Linux Security
Band 3: Analyse

Sicherheitsanalysen durchführen und Schwachstellen professionell bewerten

Von Nmap und Wireshark über Privilege Escalation und Passwort-Audits bis zu Incident Response.
Mit 36 Laborübungen im Vier-Maschinen-Labor, Kontrollfragen und Lernzielkontrollen je Kapitel.

Inhaltsverzeichnis

Vorwort 8

Die Laborumgebung dieses Bandes 8

server2 einrichten 8

Aufbau dieses Bandes 9

Kapitel 1: Grundlagen des Ethical Hacking 10

1. Einführung 10

2. Technische Grundlagen 10

3. Fachbegriffe 10

4. Praxis und Konfiguration 10

5. Sicherheitsbetrachtung 10

6. Praktische Laborübungen 11

7. Typische Fehler 11

8. Zusammenfassung 11

9. Kontrollfragen 12

10. Lernzielkontrolle 12

Kapitel 2: Angriffsflächen und Bedrohungsmodelle 13

1. Einführung 13

2. Technische Grundlagen 13

3. Fachbegriffe 13

4. Praxis und Konfiguration 13

5. Sicherheitsbetrachtung 13

6. Praktische Laborübungen 14

7. Typische Fehler 14

8. Zusammenfassung 14

9. Kontrollfragen 15

10. Lernzielkontrolle 15

Kapitel 3: Netzwerkerkundung mit Nmap 16

1. Einführung 16

2. Technische Grundlagen 16

3. Fachbegriffe 16

4. Praxis und Konfiguration 16

5. Sicherheitsbetrachtung 16

6. Praktische Laborübungen . . . 17
7. Typische Fehler . . . 17
8. Zusammenfassung . . . 17
9. Kontrollfragen . . . 18
10. Lernzielkontrolle . . . 18

Kapitel 4: Netzwerkverkehr mit Wireshark analysieren . . . 19

1. Einführung . . . 19
2. Technische Grundlagen . . . 19
3. Fachbegriffe . . . 19
4. Praxis und Konfiguration . . . 19
5. Sicherheitsbetrachtung . . . 19
6. Praktische Laborübungen . . . 20
7. Typische Fehler . . . 20
8. Zusammenfassung . . . 20
9. Kontrollfragen . . . 20
10. Lernzielkontrolle . . . 21

Kapitel 5: Paketanalysen mit tcpdump . . . 22

1. Einführung . . . 22
2. Technische Grundlagen . . . 22
3. Fachbegriffe . . . 22
4. Praxis und Konfiguration . . . 22
5. Sicherheitsbetrachtung . . . 22
6. Praktische Laborübungen . . . 23
7. Typische Fehler . . . 23
8. Zusammenfassung . . . 23
9. Kontrollfragen . . . 23
10. Lernzielkontrolle . . . 24

Kapitel 6: Schwachstellenanalyse mit Lynis . . . 25

1. Einführung . . . 25
2. Technische Grundlagen . . . 25
3. Fachbegriffe . . . 25
4. Praxis und Konfiguration . . . 25
5. Sicherheitsbetrachtung . . . 25
6. Praktische Laborübungen . . . 26
7. Typische Fehler . . . 26

8. Zusammenfassung . . . 26
9. Kontrollfragen . . . 26
10. Lernzielkontrolle . . . 27

Kapitel 7: Sicherheitsanalyse mit OpenSCAP . . . 28

1. Einführung . . . 28
2. Technische Grundlagen . . . 28
3. Fachbegriffe . . . 28
4. Praxis und Konfiguration . . . 28
5. Sicherheitsbetrachtung . . . 28
6. Praktische Laborübungen . . . 29
7. Typische Fehler . . . 29
8. Zusammenfassung . . . 29
9. Kontrollfragen . . . 30
10. Lernzielkontrolle . . . 30

Kapitel 8: Linux Privilege Escalation . . . 31

1. Einführung . . . 31
2. Technische Grundlagen . . . 31
3. Fachbegriffe . . . 31
4. Praxis und Konfiguration . . . 31
5. Sicherheitsbetrachtung . . . 31
6. Praktische Laborübungen . . . 32
7. Typische Fehler . . . 32
8. Zusammenfassung . . . 32
9. Kontrollfragen . . . 33
10. Lernzielkontrolle . . . 33

Kapitel 9: Linux Rootkits und Malware . . . 34

1. Einführung . . . 34
2. Technische Grundlagen . . . 34
3. Fachbegriffe . . . 34
4. Praxis und Konfiguration . . . 34
5. Sicherheitsbetrachtung . . . 34
6. Praktische Laborübungen . . . 35
7. Typische Fehler . . . 35
8. Zusammenfassung . . . 35
9. Kontrollfragen . . . 35

10. Lernzielkontrolle . . . 36

Kapitel 10: Passwortsicherheit und Hashverfahren . . . 37

1. Einführung . . . 37
2. Technische Grundlagen . . . 37
3. Fachbegriffe . . . 37
4. Praxis und Konfiguration . . . 37
5. Sicherheitsbetrachtung . . . 37
6. Praktische Laborübungen . . . 38
7. Typische Fehler . . . 38
8. Zusammenfassung . . . 38
9. Kontrollfragen . . . 38
10. Lernzielkontrolle . . . 39

Kapitel 11: Sicherheitsanalyse mit John the Ripper . . . 40

1. Einführung . . . 40
2. Technische Grundlagen . . . 40
3. Fachbegriffe . . . 40
4. Praxis und Konfiguration . . . 40
5. Sicherheitsbetrachtung . . . 40
6. Praktische Laborübungen . . . 41
7. Typische Fehler . . . 41
8. Zusammenfassung . . . 41
9. Kontrollfragen . . . 41
10. Lernzielkontrolle . . . 42

Kapitel 12: Sicherheitsanalyse mit Hashcat . . . 43

1. Einführung . . . 43
2. Technische Grundlagen . . . 43
3. Fachbegriffe . . . 43
4. Praxis und Konfiguration . . . 43
5. Sicherheitsbetrachtung . . . 43
6. Praktische Laborübungen . . . 44
7. Typische Fehler . . . 44
8. Zusammenfassung . . . 44
9. Kontrollfragen . . . 44
10. Lernzielkontrolle . . . 45

Kapitel 13: Grundlagen von Metasploit . . . 46

1. Einführung . . . 46
2. Technische Grundlagen . . . 46
3. Fachbegriffe . . . 46
4. Praxis und Konfiguration . . . 46
5. Sicherheitsbetrachtung . . . 46
6. Praktische Laborübungen . . . 47
7. Typische Fehler . . . 47
8. Zusammenfassung . . . 47
9. Kontrollfragen . . . 47
10. Lernzielkontrolle . . . 48

Kapitel 14: Webserver- und Anwendungssicherheit . . . 49

1. Einführung . . . 49
2. Technische Grundlagen . . . 49
3. Fachbegriffe . . . 49
4. Praxis und Konfiguration . . . 49
5. Sicherheitsbetrachtung . . . 49
6. Praktische Laborübungen . . . 50
7. Typische Fehler . . . 50
8. Zusammenfassung . . . 50
9. Kontrollfragen . . . 50
10. Lernzielkontrolle . . . 51

Kapitel 15: Incident Response . . . 52

1. Einführung . . . 52
2. Technische Grundlagen . . . 52
3. Fachbegriffe . . . 52
4. Praxis und Konfiguration . . . 52
5. Sicherheitsbetrachtung . . . 52
6. Praktische Laborübungen . . . 53
7. Typische Fehler . . . 53
8. Zusammenfassung . . . 53
9. Kontrollfragen . . . 53
10. Lernzielkontrolle . . . 54

Kapitel 16: Forensische Grundlagen . . . 55

1. Einführung . . . 55
2. Technische Grundlagen . . . 55

3. Fachbegriffe . . . 55
4. Praxis und Konfiguration . . . 55
5. Sicherheitsbetrachtung . . . 55
6. Praktische Laborübungen . . . 56
7. Typische Fehler . . . 56
8. Zusammenfassung . . . 56
9. Kontrollfragen . . . 57
10. Lernzielkontrolle . . . 57

Kapitel 17: Security Auditing . . . 58

1. Einführung . . . 58
2. Technische Grundlagen . . . 58
3. Fachbegriffe . . . 58
4. Praxis und Konfiguration . . . 58
5. Sicherheitsbetrachtung . . . 58
6. Praktische Laborübungen . . . 59
7. Typische Fehler . . . 59
8. Zusammenfassung . . . 59
9. Kontrollfragen . . . 59
10. Lernzielkontrolle . . . 60

Kapitel 18: Professionelle Sicherheitsberichte . . . 61

1. Einführung . . . 61
2. Technische Grundlagen . . . 61
3. Fachbegriffe . . . 61
4. Praxis und Konfiguration . . . 61
5. Sicherheitsbetrachtung . . . 61
6. Praktische Laborübungen . . . 62
7. Typische Fehler . . . 62
8. Zusammenfassung . . . 62
9. Kontrollfragen . . . 63
10. Lernzielkontrolle . . . 63
Abschlussprojekt Band 3: Linux Security Audit . . . 63

Verzeichnis der Befehlsbeispiele . . . 64

Index . . . 65

Vorwort

Band 1 hat einen Grundserver aufgebaut, Band 2 ihn professionell gehärtet. Band 3 wechselt die Perspektive: Es prüft, ob diese Härtung tatsächlich wirkt, indem es dieselben Methoden anwendet, mit denen auch ein Angreifer vorgehen würde, ausschließlich im eigenen, isolierten Labor. Der Bogen reicht von der Netzwerkerkundung über Passwort-Audits und ein Exploitation-Framework bis zur forensischen Nachbereitung eines Vorfalls und einem professionellen Abschlussbericht.

Dieses Buch setzt Band 1 und 2 als bekannt voraus. Es behandelt dieselben Werkzeuge wie eine parallele, auf einzelne Tools fokussierte Reihe (hping3, Nmap, Wireshark, John the Ripper, Hashcat, Metasploit), hier aber eingebettet in die zehnteilige Fachbuchstruktur dieser Reihe und konsequent auf den in Band 1 und 2 aufgebauten Server bezogen.

Die Laborumgebung dieses Bandes

Band 3 vervollständigt das Labor aus dem Master-Prompt um die beiden fehlenden Maschinen:

System	Rolle	Betriebssystem
security-client	Analysewerkstatt (aus Band 1/2 bekannt)	Kali Linux
server1	gehärteter Server aus Band 1/2, jetzt Prüfobjekt	Debian 13
server2	bewusst einfach konfigurierter Testserver, neu in diesem Band	Debian 13
client1	Windows-11-Client für plattformübergreifende Übungen	Windows 11

Beispiel V.1: Erweiterte Netzwerkarchitektur

```
security-client   192.168.56.10   (Kali Linux)
server1           192.168.56.20   (Debian 13, gehärtet)
server2           192.168.56.30   (Debian 13, bewusst einfach konfiguriert)
client1           192.168.56.40   (Windows 11)
Alle vier Systeme im selben, isolierten Internal Network ohne Internetzugang.
```

> **WARNUNG – Sicherheitsrisiko**
>
> Jede in diesem Buch gezeigte Analyse- oder Angriffstechnik ist ausschließlich für den Einsatz gegen `server1`, `server2` und `client1` in diesem isolierten Labor bestimmt, oder im Rahmen eines schriftlich autorisierten, realen Auftrags. Der Einsatz gegen fremde, nicht ausdrücklich freigegebene Systeme ist nach §§ 202a-202c sowie § 303b StGB strafbar, unabhängig vom Ergebnis.

server2 einrichten

`server2` dient als bewusst einfach konfiguriertes Übungsziel, damit Erkundung und Schwachstellenanalyse nicht am bereits gehärteten `server1` ins Leere laufen.

Beispiel V.2: server2 einrichten (Kurzübersicht)

```
1. Debian 13 Minimalinstallation wie server1 (Vorwort Band 1)
2. Bewusst zusätzliche, einfache Dienste installieren, z. B.:
   $ sudo apt install vsftpd apache2
3. Keine der Härtungsmaßnahmen aus Band 2 anwenden
4. Netzwerkadapter auf dasselbe Internal Network wie die übrigen Systeme
```

Aufbau dieses Bandes

Achtzehn Kapitel, gegliedert in fünf Abschnitte: Grundlagen und Methodik (1-2), Netzwerk- und Systemanalyse (3-9), Passwortsicherheit (10-12), Exploitation und Anwendungssicherheit (13-14), sowie Reaktion und Berichtswesen (15-18).

Kapitel 1: Grundlagen des Ethical Hacking

Lernziele

- Du unterscheidest White-, Grey- und Black-Hat-Hacking nach Autorisierung.
- Du benennst die rechtlichen Grundlagen für autorisierte Sicherheitstests in Deutschland.
- Du kennst den Aufbau eines Penetrationstest-Vertrags in Grundzügen.

1. Einführung

Jede Technik in diesem Buch gleicht technisch dem, was ein Angreifer tun würde. Der einzige Unterschied ist die Autorisierung. Dieses Kapitel legt fest, was das konkret bedeutet, bevor Kapitel 3 die erste praktische Technik einführt.

2. Technische Grundlagen

Ethical Hacking bezeichnet die autorisierte Anwendung von Angriffstechniken, um Schwachstellen vor einem tatsächlichen Angreifer zu finden. Die Unterscheidung in **White Hat** (autorisiert), **Grey Hat** (ohne klare Autorisierung, aber ohne böswillige Absicht) und **Black Hat** (unautorisiert und böswillig) ist kein technischer, sondern ein rechtlicher und ethischer Unterschied.

Beispiel 1.1: Derselbe Scan, drei rechtliche Einordnungen (Konzept)

```
White Hat:  Scan gegen server2, mit Vertrag und definiertem Scope
Grey Hat:   Scan gegen ein fremdes System ohne Auftrag, Ergebnis gemeldet
Black Hat:  Scan gegen ein fremdes System ohne Auftrag, ausgenutzt
```

3. Fachbegriffe

Ein **Scope** legt vertraglich fest, welche Systeme ein Test umfassen darf und welche ausdrücklich ausgeschlossen sind. Eine **Freistellungserklärung** (Get-out-of-jail-free letter) bestätigt schriftlich die Autorisierung und kann im Ernstfall gegenüber Behörden vorgelegt werden. §§ 202a-202c StGB (Ausspähen, Abfangen und Vorbereiten des Ausspähens von Daten) bilden den zentralen strafrechtlichen Rahmen in Deutschland; § 303b StGB erfasst zusätzlich Computersabotage.

4. Praxis und Konfiguration

Beispiel 1.2: Mindestinhalt eines Scope-Dokuments

```
1. Zielsysteme (IP-Adressen/Hostnamen), explizit ausgeschlossene Systeme
2. Zeitfenster des Tests
3. Erlaubte und verbotene Methoden (z. B. Denial-of-Service: ja/nein)
4. Notfallkontakt beim Auftraggeber
5. Vertraulichkeitsregeln für gefundene Daten
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Es gibt keine Technik in diesem Buch, die „nur für legale Zwecke" reserviert ist. Jede hier gezeigte Methode ist identisch mit dem, was auch ein unautorisierter Angreifer einsetzen würde. Prüfe vor jeder Übung in diesem Buch, ob du dich tatsächlich innerhalb des in diesem Vorwort definierten Labors bewegst.

6. Praktische Laborübungen

Übung A: Einen Mini-Scope für das eigene Labor formulieren

Aufgabenstellung: Formuliere ein Scope-Dokument nach Beispiel 1.2 für die Übungen dieses Buches. **Ausgangssituation:** Das Labor aus dem Vorwort ist eingerichtet. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** Beispiel 1.2 mit den konkreten Angaben aus dem Vorwort dieses Bandes ausfüllen. **Erwartetes Ergebnis:** Ein kurzes, vollständiges Dokument mit allen fünf Pflichtangaben. **Überprüfung:** Jeder Punkt aus Beispiel 1.2 ist konkret, nicht nur platzhalterhaft, ausgefüllt. **Fehleranalyse:** Unklare Angabe? Mit den tatsächlichen IP-Adressen aus dem Vorwort abgleichen. **Musterlösung:** Ein vollständiges Scope-Dokument, das `server1` und `server2` als Ziel, `client1` als optionales Analyseziel und das externe Internet als ausdrücklich ausgeschlossen benennt.

Übung B: Einordnung realer Szenarien

Aufgabenstellung: Ordne drei kurze, fiktive Szenarien als White, Grey oder Black Hat ein und begründe. **Ausgangssituation:** keine. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** Drei Szenarien selbst formulieren oder aus einer Fachquelle entnehmen, jeweils nach Autorisierung bewerten. **Erwartetes Ergebnis:** Eine begründete Einordnung je Szenario. **Überprüfung:** Die Begründung bezieht sich auf die Autorisierung, nicht auf die eingesetzte Technik. **Fehleranalyse:** Einordnung nach Technik statt Autorisierung vorgenommen? Zurück zu Beispiel 1.1. **Musterlösung:** Eine korrekte Lösung erkennt, dass identische Technik je nach Autorisierung alle drei Kategorien ergeben kann.

7. Typische Fehler

Fehler	Ursache	Behebung
Technik statt Autorisierung als Unterscheidungsmerkmal	Verwechslung von Mittel und Rahmen	immer zuerst nach Autorisierung fragen
Kein schriftlicher Scope vor Testbeginn	Bequemlichkeit, mündliche Absprache als ausreichend angenommen	Scope immer schriftlich festhalten
Eigenes Labor für „sowieso erlaubt" gehalten	fehlende Dokumentation der Autorisierung	auch im eigenen Labor Vorgehen dokumentieren

8. Zusammenfassung

Zusammenfassung

- White, Grey und Black Hat unterscheiden sich durch Autorisierung, nicht durch Technik.
- §§ 202a-202c und § 303b StGB bilden den zentralen strafrechtlichen Rahmen in Deutschland.
- Ein schriftlicher Scope ist die Voraussetzung jedes autorisierten Tests.
- Dieses Buch arbeitet ausschließlich im isolierten, eigenen Vier-Maschinen-Labor.

9. Kontrollfragen

1. Was unterscheidet White Hat von Black Hat? (leicht)
2. Welche Paragrafen bilden den zentralen strafrechtlichen Rahmen? (leicht)
3. Was gehört mindestens in ein Scope-Dokument? (mittel)
4. Wozu dient eine Freistellungserklärung? (mittel)
5. Ein Kollege möchte „nur kurz" ein System außerhalb des vereinbarten Scopes mittesten, weil es „offensichtlich zum selben Projekt gehört". Wie reagierst du? (schwer)

Lösungshinweise: 1. Ausschließlich die Autorisierung, nicht die eingesetzte Technik. 2. §§ 202a-202c StGB und § 303b StGB. 3. Zielsysteme, Zeitfenster, erlaubte Methoden, Notfallkontakt, Vertraulichkeitsregeln. 4. Sie belegt im Ernstfall gegenüber Behörden die Autorisierung des Tests. 5. Ablehnen und auf die schriftliche Scope-Erweiterung bestehen, auch wenn die Zugehörigkeit naheliegend erscheint – Autorisierung muss ausdrücklich, nicht vermutet vorliegen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] White, Grey und Black Hat anhand der Autorisierung unterscheiden kannst,
- [ ] ein vollständiges Scope-Dokument für das eigene Labor erstellt hast,
- [ ] die zentralen Paragrafen benennen kannst,
- [ ] auf eine Scope-Überschreitung angemessen reagieren würdest.

Kapitel 2: Angriffsflächen und Bedrohungsmodelle

Lernziele

- Du erstellst ein einfaches Bedrohungsmodell für ein konkretes System.
- Du ordnest Bedrohungen nach der STRIDE-Methode ein.
- Du leitest aus einem Bedrohungsmodell konkrete Testschwerpunkte ab.

1. Einführung

Band 1, Kapitel 1 hat die Angriffsfläche als Konzept eingeführt. Dieses Kapitel macht daraus ein systematisches Werkzeug: ein **Bedrohungsmodell**, das vor jeder praktischen Analyse entsteht und festlegt, wonach überhaupt gesucht werden soll.

2. Technische Grundlagen

Ein Bedrohungsmodell beschreibt ein System, seine Vertrauensgrenzen (wo wechselt die Kontrolle zwischen Akteuren) und die Bedrohungen, die an jeder dieser Grenzen plausibel sind. **STRIDE** ist eine verbreitete Kategorisierung: Spoofing (Identitätsvortäuschung), Tampering (Manipulation), Repudiation (Abstreitbarkeit), Information Disclosure (Offenlegung), Denial of Service, Elevation of Privilege.

Beispiel 2.1: Eine Vertrauensgrenze (Konzept)

```
Internet <-- Vertrauensgrenze --> server2 (Webserver) <-- Vertrauensgrenze --> Datenbank
```

3. Fachbegriffe

Eine **Vertrauensgrenze** markiert einen Punkt, an dem Daten oder Kontrolle zwischen unterschiedlich vertrauenswürdigen Bereichen wechseln. Ein **Angriffsvektor** ist der konkrete Weg, über den eine Bedrohung ein System erreicht.

4. Praxis und Konfiguration

Beispiel 2.2: STRIDE auf server2 angewendet (Auszug)

```
S - Spoofing: gefälschte Absenderadresse bei FTP-Zugriff
T - Tampering: Veränderung hochgeladener Dateien
I - Info Disclosure: Verzeichnislisting durch Webserver-Fehlkonfiguration
D - Denial of Service: Ressourcenerschöpfung durch viele Verbindungen
E - Elevation of Privilege: Rechteausweitung über unsichere SUID-Programme
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Ein Bedrohungsmodell ist eine Prognose, keine vollständige Garantie. Es lenkt die Analyse auf plausible Schwerpunkte, ersetzt aber nicht die tatsächliche, praktische Prüfung in den folgenden Kapiteln.

6. Praktische Laborübungen

Übung A: Ein Bedrohungsmodell für server2 erstellen

Aufgabenstellung: Erstelle ein STRIDE-Modell für `server2` nach Beispiel 2.2. **Ausgangssituation:** `server2` nach Vorwort-Installation. **Voraussetzungen:** Kenntnis der auf `server2` installierten Dienste. **Schritt-für-Schritt-Anleitung:** Für jede STRIDE-Kategorie mindestens eine plausible Bedrohung für `server2` formulieren. **Erwartetes Ergebnis:** Sechs benannte Bedrohungen, eine je Kategorie. **Überprüfung:** Jede Bedrohung bezieht sich auf einen tatsächlich auf `server2` vorhandenen Dienst. **Fehleranalyse:** Bedrohung zu allgemein formuliert? Auf eine konkrete, tatsächlich vorhandene Komponente beziehen. **Musterlösung:** Ein vollständiges Modell mit sechs konkreten, auf `server2` bezogenen Bedrohungen.

Übung B: Testschwerpunkte ableiten

Aufgabenstellung: Leite aus dem Bedrohungsmodell aus Übung A drei konkrete Testschwerpunkte für die folgenden Kapitel ab. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** Die plausibelsten drei Bedrohungen auswählen, jeweils einem späteren Kapitel dieses Buches zuordnen (z. B. Elevation of Privilege → Kapitel 8). **Erwartetes Ergebnis:** Eine priorisierte Liste mit Kapitelzuordnung. **Überprüfung:** Jeder Schwerpunkt lässt sich einem konkreten, später behandelten Werkzeug zuordnen. **Fehleranalyse:** Keine passende Zuordnung? Inhaltsverzeichnis dieses Bandes konsultieren. **Musterlösung:** Eine vollständige Lösung verbindet jede der drei Bedrohungen mit einem konkreten, später folgenden Kapitel.

7. Typische Fehler

Fehler	Ursache	Behebung
Bedrohungsmodell zu abstrakt	keine konkreten Systemdetails einbezogen	immer auf tatsächliche Dienste und Konfigurationen beziehen
STRIDE-Kategorien übersprungen	unvollständige, einseitige Betrachtung	alle sechs Kategorien mindestens einmal prüfen
Modell nie mit der Praxis abgeglichen	Theorie und Übung getrennt behandelt	Modell aktiv zur Priorisierung der folgenden Kapitel nutzen

8. Zusammenfassung

Zusammenfassung

- Ein Bedrohungsmodell beschreibt System, Vertrauensgrenzen und plausible Bedrohungen.
- STRIDE strukturiert Bedrohungen in sechs Kategorien.
- Ein Modell lenkt die Analyse, ersetzt aber nicht die praktische Prüfung.
- Aus dem Modell lassen sich konkrete Testschwerpunkte für die folgenden Kapitel ableiten.

9. Kontrollfragen

1. Wofür steht STRIDE? (leicht)
2. Was ist eine Vertrauensgrenze? (leicht)
3. Warum ersetzt ein Bedrohungsmodell nicht die praktische Prüfung? (mittel)
4. Was ist ein Angriffsvektor? (mittel)
5. Zwei Systeme haben identische Software, aber unterschiedliche Vertrauensgrenzen (eines ist nur intern erreichbar, eines öffentlich). Sollte ihr Bedrohungsmodell identisch sein? (schwer)

Lösungshinweise: 1. Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege. 2. Ein Punkt, an dem Daten oder Kontrolle zwischen unterschiedlich vertrauenswürdigen Bereichen wechseln. 3. Weil es eine Prognose ist, keine vollständige Garantie gegen tatsächliche, noch unbekannte Schwachstellen. 4. Der konkrete Weg, über den eine Bedrohung ein System erreicht. 5. Nein, die unterschiedliche Erreichbarkeit verändert die Vertrauensgrenzen und damit die Wahrscheinlichkeit einzelner Bedrohungen erheblich.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] ein STRIDE-Modell für ein konkretes System erstellt hast,
- [ ] Vertrauensgrenzen in einem System benennen kannst,
- [ ] aus einem Modell konkrete Testschwerpunkte ableitest,
- [ ] erklären kannst, warum identische Software unterschiedliche Bedrohungsmodelle haben kann.

Kapitel 3: Netzwerkerkundung mit Nmap

Lernziele

- Du führst einen SYN-Scan gegen server2 durch und interpretierst das Ergebnis.
- Du setzt Nmap-Skripte (NSE) für Diensterkennung ein.
- Du vergleichst die Scan-Ergebnisse von server1 und server2.

1. Einführung

Nachdem Kapitel 2 festgelegt hat, wonach gesucht wird, beginnt die praktische Analyse mit der Netzwerkebene. **Nmap** automatisiert, was Band 1/2 bereits mit `ss -tulpen` lokal gezeigt haben, von außen betrachtet.

2. Technische Grundlagen

Nmap sendet gezielt konstruierte Pakete und wertet die Antworten aus, um offene Ports, laufende Dienste und teilweise das Betriebssystem zu identifizieren. Ein **SYN-Scan** (`-sS`) sendet nur den ersten Schritt des TCP-Handshakes und wertet die Antwort aus, ohne die Verbindung vollständig aufzubauen.

Beispiel 3.1: SYN-Scan mit Versionserkennung

```
$ sudo nmap -sS -sV -p 1-1000 192.168.56.30
```

3. Fachbegriffe

Die **Nmap Scripting Engine (NSE)** erweitert Nmap um Lua-Skripte für protokollspezifische Prüfungen, von einfachen Bannerabfragen bis zu Schwachstellenchecks. `-A` bündelt Versions-, Betriebssystemerkennung und Standard-NSE-Skripte in einem Aufruf.

4. Praxis und Konfiguration

Beispiel 3.2: Gezieltes NSE-Skript gegen einen erkannten Dienst

```
$ sudo nmap --script=ftp-anon -p 21 192.168.56.30
```

Beispiel 3.3: Ergebnis dokumentieren

```
$ sudo nmap -A -oA server2_scan 192.168.56.30
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Scanne ausschließlich `server1` und `server2` in deinem eigenen Labor. Ein Scan gegen ein fremdes System ist eine aktive, unautorisierte Kontaktaufnahme, unabhängig vom Ergebnis.

Der Vergleich zwischen `server1` (gehärtet, Band 2) und `server2` (unverändert) macht den praktischen Wert der Härtung sichtbar: `server1` sollte deutlich weniger offene Ports und weniger auswertbare Bannerinformationen zeigen.

6. Praktische Laborübungen

Übung A: server2 vollständig scannen

Aufgabenstellung: Führe einen vollständigen SYN-Scan mit Versionserkennung gegen `server2` durch. **Ausgangssituation:** `server2` nach Vorwort-Installation. **Voraussetzungen:** Nmap auf `security-client`. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 3.1. **Erwartetes Ergebnis:** Mehrere offene Ports mit erkannten Diensten (FTP, HTTP). **Überprüfung:** Die erkannten Dienste stimmen mit der Installation aus dem Vorwort überein. **Fehleranalyse:** Keine Ergebnisse? Netzwerkverbindung und Firewall-Status von `security-client` prüfen. **Musterlösung:** Eine vollständige Liste offener Ports mit Dienst- und Versionsangabe.

Übung B: server1 und server2 vergleichen

Aufgabenstellung: Wiederhole den Scan gegen `server1` und vergleiche das Ergebnis. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** Beispiel 3.1 gegen `192.168.56.20` wiederholen, Ergebnisse gegenüberstellen. **Erwartetes Ergebnis:** `server1` zeigt deutlich weniger offene Ports. **Überprüfung:** Die Differenz lässt sich auf konkrete Band-2-Maßnahmen zurückführen (Firewall, deaktivierte Dienste). **Fehleranalyse:** Kein Unterschied sichtbar? Band-2-Firewallregeln auf `server1` erneut prüfen. **Musterlösung:** Eine Tabelle, die für jeden Port die Differenz zwischen beiden Servern benennt und begründet.

7. Typische Fehler

Fehler	Ursache	Behebung
Scan gegen falsches Ziel	IP-Adresse verwechselt	Adressen aus dem Vorwort vor jedem Scan prüfen
Ergebnis ohne Dokumentation verworfen	`-oA` vergessen	jeden relevanten Scan dokumentiert speichern
Unterschied zwischen Servern nicht erklärt	Band-2-Maßnahmen nicht mehr präsent	Band 2 bei Bedarf gezielt nachschlagen

8. Zusammenfassung

Zusammenfassung

- Nmap automatisiert Port-, Dienst- und Versionserkennung aus der Ferne.
- NSE-Skripte prüfen gezielt einzelne, erkannte Dienste.
- Der Vergleich zwischen gehärtetem und ungehärtetem Server macht Band-2-Maßnahmen messbar.
- Jeder Scan gehört dokumentiert, ausschließlich gegen die eigenen Laborsysteme.

9. Kontrollfragen

1. Was macht ein SYN-Scan? (leicht)
2. Wofür steht NSE? (leicht)
3. Was bündelt die Option `-A`? (mittel)
4. Warum sollte jeder Scan dokumentiert werden? (mittel)
5. `server1` zeigt bei gleichem Scan deutlich weniger offene Ports als `server2`. Nenne zwei mögliche, aus Band 2 bekannte Ursachen. (schwer)

Lösungshinweise: 1. Er sendet nur den ersten Schritt des TCP-Handshakes, ohne die Verbindung vollständig aufzubauen. 2. Nmap Scripting Engine. 3. Versions-, Betriebssystemerkennung und Standard-NSE-Skripte. 4. Um Ergebnisse später nachvollziehen und mit folgenden Scans vergleichen zu können. 5. Die Firewall-Konfiguration aus Band 2 (Kapitel 4/5) und die Deaktivierung nicht benötigter Dienste (Band 2, Kapitel 8).

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen vollständigen Scan gegen `server2` durchgeführt hast,
- [ ] ein NSE-Skript gezielt eingesetzt hast,
- [ ] die Ergebnisse von `server1` und `server2` begründet verglichen hast,
- [ ] Scan-Ergebnisse dokumentiert gespeichert hast.

Kapitel 4: Netzwerkverkehr mit Wireshark analysieren

Lernziele

- Du schneidest Netzwerkverkehr gezielt mit Capture-Filtern mit.
- Du folgst einem TCP-Stream und liest Klartext-Zugangsdaten.
- Du erkennst den Unterschied zu verschlüsseltem Verkehr.

1. Einführung

Nmap (Kapitel 3) zeigt, welche Dienste erreichbar sind. **Wireshark** zeigt, was tatsächlich über die Leitung geht – die notwendige Ergänzung, um zu verstehen, was ein gefundener Dienst konkret preisgibt.

2. Technische Grundlagen

Wireshark schneidet Verkehr an einer Netzwerkschnittstelle mit und stellt ihn in OSI-Schichten dar. **Capture-Filter** (BPF-Syntax) entscheiden beim Mitschnitt, was überhaupt aufgezeichnet wird; **Display-Filter** blenden nachträglich aus bereits Aufgezeichnetem aus.

Beispiel 4.1: Nur FTP-Verkehr zu server2 mitschneiden

```
$ sudo tshark -i eth0 -f "host 192.168.56.30 and port 21" -w ftp_mitschnitt.pcapng
```

3. Fachbegriffe

Follow TCP Stream fasst alle Pakete einer Verbindung zu einem lesbaren Text zusammen. Klartext-Protokolle wie FTP und unverschlüsseltes HTTP übertragen Zugangsdaten ohne jeden Schutz; TLS-verschlüsselter Verkehr zeigt stattdessen unlesbare „Application Data".

4. Praxis und Konfiguration

Beispiel 4.2: Mitschnitt in der GUI analysieren

```
Datei öffnen -> Display-Filter "ftp" -> Paket auswählen
-> Rechtsklick -> "Follow" -> "TCP Stream"
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Schneide ausschließlich Verkehr in deinem eigenen, isolierten Labor mit. Das unbefugte Abfangen fremder Datenübermittlung ist nach § 202b StGB strafbar, unabhängig von der Absicht.

Ein FTP-Login gegen `server2` zeigt Benutzername und Passwort im Klartext – der direkte, praktische Beleg dafür, warum Band 1, Kapitel 10 TLS-verschlüsselte Alternativen empfiehlt.

6. Praktische Laborübungen

Übung A: FTP-Anmeldedaten im Klartext mitschneiden

Aufgabenstellung: Schneide eine FTP-Anmeldung an `server2` mit und lies die Zugangsdaten im Stream. **Ausgangssituation:** `server2` mit laufendem FTP-Dienst. **Voraussetzungen:** Mitschnitt-Rechte auf `security-client`. **Schritt-für-Schritt-Anleitung:** Mitschnitt starten (Beispiel 4.1), von einem zweiten Terminal `ftp 192.168.56.30` mit Testzugangsdaten durchführen, Mitschnitt stoppen, Stream folgen. **Erwartetes Ergebnis:** Benutzername und Passwort erscheinen lesbar im Stream. **Überprüfung:** Die gelesenen Zugangsdaten stimmen mit den tatsächlich eingegebenen überein. **Fehleranalyse:** Kein FTP-Verkehr sichtbar? Capture-Filter und Portnummer (21) prüfen. **Musterlösung:** Der vollständige FTP-Dialog inklusive lesbarer Zugangsdaten im Stream-Fenster.

Übung B: Verschlüsselten Vergleichsverkehr beobachten

Aufgabenstellung: Schneide eine SSH-Verbindung zu `server1` mit und vergleiche die Lesbarkeit mit Übung A. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** Mitschnitt auf Port 22 statt 21, SSH-Verbindung aufbauen, Stream folgen. **Erwartetes Ergebnis:** Der Stream zeigt keine lesbaren Zugangsdaten, nur verschlüsselte Daten. **Überprüfung:** Kein Klartext-Passwort im Stream auffindbar. **Fehleranalyse:** Scheinbar lesbare Inhalte? Es handelt sich um Protokoll-Metadaten vor der eigentlichen Verschlüsselung, nicht um Nutzdaten. **Musterlösung:** Ein klarer Kontrast zwischen lesbarem FTP- und unlesbarem SSH-Stream, als praktischer Beleg für den Wert von Verschlüsselung.

7. Typische Fehler

Fehler	Ursache	Behebung
Capture-Filter zu eng, Verkehr fehlt	falscher Port oder falsche Adresse	Filter gegen die tatsächliche Zieladresse prüfen
Verwechslung von Capture- und Display-Filter	Syntaxunterschied unbekannt	BPF für Capture, Wireshark-Syntax für Display
Mitschnitt gegen fremdes Netzwerk gestartet	Laborgrenzen nicht geprüft	Schnittstelle vor jedem Mitschnitt verifizieren

8. Zusammenfassung

Zusammenfassung

- Capture-Filter entscheiden beim Mitschnitt, Display-Filter danach.
- Follow TCP Stream macht eine ganze Verbindung zusammenhängend lesbar.
- Klartext-Protokolle wie FTP zeigen Zugangsdaten offen, TLS-Verkehr nicht.
- Jeder Mitschnitt gehört ausschließlich ins eigene, isolierte Labor.

9. Kontrollfragen

1. Was unterscheidet Capture- von Display-Filtern? (leicht)
2. Wozu dient Follow TCP Stream? (leicht)
3. Warum zeigt ein FTP-Mitschnitt Zugangsdaten im Klartext? (mittel)

4. Welcher Paragraf macht unbefugtes Mitschneiden strafbar? (mittel)
5. Ein Kollege argumentiert, ein Mitschnitt im eigenen Heimnetz sei immer erlaubt, da es „sein eigenes Netz" sei. Was entgegnest du? (schwer)

Lösungshinweise: 1. Capture-Filter bestimmen, was aufgezeichnet wird, Display-Filter blenden nachträglich aus. 2. Es fasst eine ganze TCP-Verbindung zu lesbarem Text zusammen. 3. Weil FTP keine Verschlüsselung vorsieht und Anmeldedaten unverändert überträgt. 4. § 202b StGB. 5. Auch im eigenen Netz kann Verkehr anderer Personen (Familie, Mitbewohner) mitgeschnitten werden, deren Kommunikation nicht automatisch zur eigenen Verfügung steht – Eigentum am Netz begründet nicht automatisch die Berechtigung, fremden Inhalt mitzulesen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen gezielten Mitschnitt mit Capture-Filter erstellt hast,
- [ ] Klartext-Zugangsdaten über Follow TCP Stream gefunden hast,
- [ ] den Unterschied zu verschlüsseltem Verkehr praktisch beobachtet hast,
- [ ] die rechtliche Grenze des Mitschneidens erklären kannst.

Kapitel 5: Paketanalysen mit tcpdump

Lernziele

- Du erfasst Netzwerkverkehr ohne grafische Oberfläche direkt auf einem Server.
- Du formulierst tcpdump-Filterausdrücke für gezielte Analysen.
- Du exportierst einen tcpdump-Mitschnitt für die spätere Wireshark-Analyse.

1. Einführung

Wireshark (Kapitel 4) braucht eine grafische Oberfläche. **tcpdump** erfüllt denselben Grundzweck direkt auf der Kommandozeile – wichtig für einen Server, auf dem keine grafische Umgebung läuft, etwa `server1` oder `server2` selbst.

2. Technische Grundlagen

tcpdump nutzt dieselbe BPF-Filtersyntax wie Wiresharks Capture-Filter (Kapitel 4), zeigt Pakete aber direkt im Terminal an oder schreibt sie in eine Datei im pcap-Format, das sich später mit Wireshark öffnen lässt.

Beispiel 5.1: Verkehr direkt auf server2 mitschneiden

```
$ sudo tcpdump -i eth0 port 21 -w /tmp/ftp_server2.pcap
```

3. Fachbegriffe

Das **pcap-Format** ist das gemeinsame Austauschformat für Mitschnitte zwischen tcpdump, Wireshark und vielen weiteren Werkzeugen. Die Option `-n` verzichtet auf Namensauflösung und beschleunigt die Anzeige bei großen Mitschnitten.

4. Praxis und Konfiguration

Beispiel 5.2: Verkehr direkt im Terminal lesbar anzeigen

```
$ sudo tcpdump -i eth0 -n -X port 21
```

`-X` zeigt den Paketinhalt zusätzlich in Hex- und ASCII-Darstellung.

Beispiel 5.3: Mitschnitt für spätere Analyse übertragen

```
$ scp eugen@192.168.56.30:/tmp/ftp_server2.pcap .
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

tcpdump benötigt root-Rechte für Rohsockets und erfasst sämtlichen Verkehr der gewählten Schnittstelle, nicht nur den vermeintlich interessanten. Begrenze den Umfang bewusst über Filter, statt unbegrenzt mitzuschneiden.

6. Praktische Laborübungen

Übung A: Direkt auf server2 mitschneiden

Aufgabenstellung: Schneide eine FTP-Verbindung direkt auf `server2` mit tcpdump mit. **Ausgangssituation:** `server2` mit laufendem FTP-Dienst, SSH-Zugriff darauf. **Voraussetzungen:** sudo-Rechte auf `server2`. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 5.1, parallel von `security-client` aus eine FTP-Verbindung aufbauen. **Erwartetes Ergebnis:** Eine pcap-Datei mit dem vollständigen FTP-Dialog. **Überprüfung:** `tcpdump -r /tmp/ftp_server2.pcap` zeigt die mitgeschnittenen Pakete. **Fehleranalyse:** Leere Datei? Prüfe, ob der Filter die korrekte Schnittstelle und den korrekten Port trifft. **Musterlösung:** Eine vollständige pcap-Datei mit sichtbarem Drei-Wege-Handshake und FTP-Kommandos.

Übung B: Mitschnitt exportieren und in Wireshark weiteranalysieren

Aufgabenstellung: Übertrage die Datei aus Übung A auf `security-client` und analysiere sie dort mit Wireshark. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A, SCP-Zugriff. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 5.3, danach Datei in Wireshark öffnen und Stream folgen (vergleiche Kapitel 4). **Erwartetes Ergebnis:** Dieselben, bereits aus Kapitel 4 bekannten Zugangsdaten erscheinen auch in diesem, anders erzeugten Mitschnitt. **Überprüfung:** Beide Mitschnitte (Kapitel 4 und dieses Kapitel) zeigen inhaltlich dasselbe Ergebnis. **Fehleranalyse:** Datei öffnet nicht in Wireshark? Prüfe, ob die Übertragung per `scp` vollständig und unbeschädigt war. **Musterlösung:** Eine erfolgreiche Übertragung und Analyse, die zeigt, dass tcpdump- und Wireshark-Mitschnitte austauschbar sind.

7. Typische Fehler

Fehler	Ursache	Behebung
Mitschnitt ohne Filter gestartet	Bequemlichkeit	Filter immer gezielt setzen, besonders auf produktiven Systemen
pcap-Datei lässt sich nicht öffnen	unvollständige Übertragung	Übertragung mit Prüfsumme verifizieren
Root-Rechte vergessen	Unkenntnis der Voraussetzung	tcpdump grundsätzlich mit `sudo` starten

8. Zusammenfassung

Zusammenfassung

- tcpdump erfüllt denselben Grundzweck wie Wireshark, direkt auf der Kommandozeile.
- Beide nutzen dieselbe BPF-Filtersyntax und das gemeinsame pcap-Format.
- Mitschnitte lassen sich zwischen Systemen übertragen und mit Wireshark weiteranalysieren.
- Unbegrenztes Mitschneiden ohne Filter sollte vermieden werden.

9. Kontrollfragen

1. Welches Dateiformat verbindet tcpdump und Wireshark? (leicht)

2. Wofür steht die Option `-X` bei tcpdump? (leicht)
3. Warum braucht tcpdump root-Rechte? (mittel)
4. Wann ist tcpdump gegenüber Wireshark die praktischere Wahl? (mittel)
5. Ein Mitschnitt auf einem produktiven Server soll möglichst wenig Systemlast erzeugen. Welche zwei konkreten Maßnahmen würdest du empfehlen? (schwer)

Lösungshinweise: 1. pcap. 2. Sie zeigt den Paketinhalt zusätzlich in Hex- und ASCII-Darstellung. 3. Weil es Rohsockets benötigt, die privilegierten Zugriff erfordern. 4. Wenn keine grafische Oberfläche verfügbar ist, etwa direkt auf einem Server. 5. Einen engen Filterausdruck statt unbegrenztem Mitschnitt verwenden, und die Mitschnittdauer sowie Dateigröße von vornherein begrenzen (z. B. mit `-c` für eine feste Paketanzahl).

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen gezielten Mitschnitt direkt auf einem Server erstellt hast,
- [ ] einen tcpdump-Mitschnitt erfolgreich nach Wireshark übertragen hast,
- [ ] erklären kannst, wann tcpdump gegenüber Wireshark sinnvoller ist,
- [ ] Maßnahmen zur Lastreduktion beim Mitschneiden nennen kannst.

Kapitel 6: Schwachstellenanalyse mit Lynis

Lernziele

- Du führst einen vollständigen Lynis-Audit gegen ein System durch.
- Du interpretierst den Hardening-Index und priorisierst Empfehlungen.
- Du vergleichst die Ergebnisse von server1 und server2.

1. Einführung

Band 2, Kapitel 1 hat manuell gegen einen CIS Benchmark geprüft. **Lynis** automatisiert genau diesen Abgleich und liefert einen konkreten, vergleichbaren Score.

2. Technische Grundlagen

Lynis prüft ein System gegen hunderte Einzelpunkte (Dateisystem, Authentifizierung, Netzwerk, installierte Software) und berechnet daraus einen **Hardening-Index** zwischen 0 und 100.

Beispiel 6.1: Lynis installieren und ausführen

```
$ sudo apt install lynis
$ sudo lynis audit system
```

3. Fachbegriffe

Der **Hardening-Index** ist ein grober, aggregierter Wert, keine exakte Sicherheitsgarantie. Jeder Prüfpunkt erzeugt eine **Warnung** (konkretes Problem) oder einen **Vorschlag** (mögliche Verbesserung), jeweils mit einer eigenen Kennung für Nachschlagezwecke.

4. Praxis und Konfiguration

Beispiel 6.2: Ergebnisbericht einsehen

```
$ sudo cat /var/log/lynis-report.dat | grep hardening_index
$ sudo cat /var/log/lynis.log | grep -A2 "Suggestion"
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Ein hoher Hardening-Index allein garantiert keine Sicherheit, ebenso wenig wie die vollständige CIS-Umsetzung aus Band 2, Kapitel 1. Lynis findet bekannte, strukturelle Muster, keine individuellen Logikfehler einer eigenen Anwendung.

6. Praktische Laborübungen

Übung A: Lynis gegen server1 ausführen

Aufgabenstellung: Führe einen vollständigen Audit gegen den gehärteten `server1` durch. **Ausgangssituation:** `server1` nach Band 1/2. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 6.1, 6.2. **Erwartetes Ergebnis:** Ein Hardening-Index und eine Liste von Vorschlägen. **Überprüfung:** Der Index liegt erkennbar über dem eines ungehärteten Systems. **Fehleranalyse:** Sehr niedriger Index trotz Band-2-Härtung? Einzelne Vorschläge durchgehen, ob eine Band-2-Maßnahme unvollständig umgesetzt wurde. **Musterlösung:** Ein dokumentierter Index mit den drei wichtigsten, noch offenen Vorschlägen.

Übung B: server1 und server2 vergleichen

Aufgabenstellung: Führe denselben Audit gegen `server2` durch und vergleiche beide Ergebnisse. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A, auf `server2` angewendet. **Schritt-für-Schritt-Anleitung:** Beispiel 6.1 auf `server2` wiederholen. **Erwartetes Ergebnis:** `server2` zeigt einen deutlich niedrigeren Index. **Überprüfung:** Die Differenz lässt sich auf konkrete, in Band 2 behandelte Maßnahmen zurückführen. **Fehleranalyse:** Kaum Unterschied? Prüfe, ob auf `server2` versehentlich doch Band-2-Maßnahmen angewendet wurden. **Musterlösung:** Eine Gegenüberstellung beider Indizes mit Begründung der wichtigsten Unterschiede.

7. Typische Fehler

Fehler	Ursache	Behebung
Index als alleinige Kennzahl missverstanden	fehlende Einordnung der Grenzen	Index immer zusammen mit den konkreten Einzelvorschlägen lesen
Vorschläge unpriorisiert abgearbeitet	keine Risikobewertung (Band 2, Kapitel 16)	Vorschläge nach Risikomatrix priorisieren
Audit nur einmalig durchgeführt	fehlendes Prozessverständnis	regelmäßige Audits einplanen

8. Zusammenfassung

Zusammenfassung

- Lynis automatisiert den Abgleich eines Systems gegen hunderte Härtungskriterien.
- Der Hardening-Index ist ein grober, aggregierter Wert, keine exakte Garantie.
- Der Vergleich zwischen gehärtetem und ungehärtetem System macht Band-2-Maßnahmen messbar.
- Vorschläge sollten priorisiert, nicht ungeordnet abgearbeitet werden.

9. Kontrollfragen

1. Was berechnet Lynis als Gesamtergebnis? (leicht)
2. Welcher Befehl startet einen vollständigen Audit? (leicht)
3. Warum ist ein hoher Hardening-Index keine Sicherheitsgarantie? (mittel)
4. Was ist der Unterschied zwischen einer Warnung und einem Vorschlag bei Lynis? (mittel)
5. `server1` zeigt trotz Band-2-Härtung einen niedrigeren Index als erwartet. Wie gehst du systematisch vor, um die Ursache zu finden? (schwer)

Lösungshinweise: 1. Einen Hardening-Index zwischen 0 und 100. 2. `sudo lynis audit system`. 3. Weil Lynis bekannte, strukturelle Muster prüft, aber keine individuellen Logikfehler eigener Anwendungen erkennt. 4. Eine Warnung meldet ein konkretes Problem, ein Vorschlag eine mögliche, nicht zwingend nötige Verbesserung. 5. Den Lynis-Log systematisch nach Warnungen durchgehen, jede gegen die in Band 2 dokumentierten, tatsächlich umgesetzten Maßnahmen abgleichen, und unvollständig umgesetzte Punkte gezielt nachbessern.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen vollständigen Lynis-Audit durchgeführt hast,
- [ ] den Hardening-Index und seine Grenzen richtig einordnest,
- [ ] die Ergebnisse von `server1` und `server2` begründet verglichen hast,
- [ ] Vorschläge nach Priorität statt ungeordnet abarbeiten kannst.

Kapitel 7: Sicherheitsanalyse mit OpenSCAP

Lernziele

- Du erklärst den Unterschied zwischen Lynis und einer formalen SCAP-Prüfung.
- Du führst eine OpenSCAP-Prüfung gegen ein Standardprofil durch.
- Du liest einen OpenSCAP-Ergebnisbericht in HTML-Form aus.

1. Einführung

Lynis (Kapitel 6) liefert einen pragmatischen, breiten Überblick. **OpenSCAP** verfolgt einen formaleren Ansatz: Es prüft gegen standardisierte, maschinenlesbare Sicherheitsrichtlinien (SCAP, Security Content Automation Protocol), wie sie etwa Behörden oder regulierte Branchen voraussetzen.

2. Technische Grundlagen

SCAP definiert ein standardisiertes Format für Sicherheitsrichtlinien und deren automatisierte Prüfung. OpenSCAP führt ein solches Regelwerk (**Profile**) gegen ein System aus und erzeugt einen detaillierten, nachvollziehbaren Bericht.

Beispiel 7.1: OpenSCAP installieren und verfügbare Profile anzeigen

```
$ sudo apt install openscap-scanner scap-security-guide
$ oscap info /usr/share/xml/scap/ssg/content/ssg-debian13-ds.xml
```

3. Fachbegriffe

Ein **Profile** ist eine konkrete, benannte Zusammenstellung von Prüfregeln innerhalb eines SCAP-Datenstroms. Ein **XCCDF-Bericht** ist das standardisierte Ergebnisformat einer SCAP-Prüfung, meist zusätzlich als lesbares HTML exportiert.

4. Praxis und Konfiguration

Beispiel 7.2: Eine Prüfung gegen ein Standardprofil durchführen

```
$ sudo oscap xccdf eval \
    --profile standard \
    --results ergebnis.xml \
    --report bericht.html \
    /usr/share/xml/scap/ssg/content/ssg-debian13-ds.xml
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

OpenSCAP-Profile sind oft auf bestimmte, regulierte Umgebungen zugeschnitten und können für ein einfaches Übungslabor unpassend strenge oder irrelevante Regeln enthalten. Lies jede als „failed" markierte Regel inhaltlich, statt den Bericht nur nach Prozentzahl zu bewerten.

6. Praktische Laborübungen

Übung A: Eine OpenSCAP-Prüfung gegen server1 durchführen

Aufgabenstellung: Führe die Prüfung aus Beispiel 7.2 gegen `server1` durch. **Ausgangssituation:** `server1` nach Band 1/2. **Voraussetzungen:** sudo-Rechte, `scap-security-guide` installiert. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 7.1, 7.2. **Erwartetes Ergebnis:** Eine HTML-Datei mit detailliertem Prüfbericht. **Überprüfung:** Der Bericht lässt sich im Browser öffnen und zeigt Pass-/Fail-Status je Regel. **Fehleranalyse:** Kein passendes Profil gefunden? Mit `oscap info` die für Debian 13 tatsächlich verfügbaren Profile auflisten. **Musterlösung:** Ein vollständiger HTML-Bericht mit nachvollziehbarer Pass/Fail-Verteilung.

Übung B: Drei fehlgeschlagene Regeln inhaltlich bewerten

Aufgabenstellung: Wähle drei als „failed" markierte Regeln aus dem Bericht und bewerte sie inhaltlich. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** Drei Regeln im HTML-Bericht auswählen, deren Beschreibung lesen, bewerten ob die Regel im Laborkontext relevant ist. **Erwartetes Ergebnis:** Für jede Regel eine begründete Einstufung: relevant und zu beheben, oder bewusst im Laborkontext zu vernachlässigen. **Überprüfung:** Die Begründung bezieht sich auf den konkreten Regelinhalt, nicht nur auf den Status. **Fehleranalyse:** Unklare Regelbeschreibung? Zusätzliche Erläuterung im Bericht oder der SCAP-Dokumentation konsultieren. **Musterlösung:** Drei begründete Einstufungen, die zwischen Laborrelevanz und tatsächlichem Produktivrisiko unterscheiden.

7. Typische Fehler

Fehler	Ursache	Behebung
Bericht nur nach Prozentzahl bewertet	fehlende inhaltliche Prüfung	jede Fail-Regel einzeln lesen
Falsches Profil für die Umgebung gewählt	Profile ungeprüft übernommen	verfügbare Profile vorher mit `oscap info` sichten
SCAP mit Lynis verwechselt	ähnlicher Zweck, unterschiedlicher Formalisierungsgrad	SCAP für standardisierte, Lynis für pragmatische Prüfungen einordnen

8. Zusammenfassung

Zusammenfassung

- OpenSCAP prüft formal gegen standardisierte SCAP-Profile.
- Profile bündeln konkrete Regeln für bestimmte Einsatzkontexte.
- Ergebnisse liegen als XCCDF-Daten und lesbarer HTML-Bericht vor.
- Jede Fail-Regel verdient inhaltliche Prüfung statt bloßer Prozentbewertung.

9. Kontrollfragen

1. Wofür steht SCAP? (leicht)
2. Welcher Befehl listet verfügbare Profile auf? (leicht)
3. Was unterscheidet OpenSCAP von Lynis im Formalisierungsgrad? (mittel)
4. Was ist ein XCCDF-Bericht? (mittel)
5. Ein Profil markiert eine Regel als „failed", die im eigenen Übungslabor keine praktische Relevanz hat. Wie gehst du damit um, ohne den Bericht zu verfälschen? (schwer)

Lösungshinweise: 1. Security Content Automation Protocol. 2. `oscap info <datei>`. 3. OpenSCAP prüft formal gegen standardisierte, oft regulatorisch vorgeschriebene Profile, Lynis liefert einen pragmatischen, breiteren Überblick. 4. Das standardisierte Ergebnisformat einer SCAP-Prüfung. 5. Die Regel im Bericht als „failed" belassen (nicht manipulieren), aber in der eigenen Dokumentation als bewusst akzeptiertes, laborspezifisches Restrisiko vermerken (vergleiche Band 2, Kapitel 16).

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] eine vollständige OpenSCAP-Prüfung durchgeführt hast,
- [ ] einen HTML-Bericht erfolgreich ausgewertet hast,
- [ ] mindestens drei Fail-Regeln inhaltlich bewertet hast,
- [ ] OpenSCAP und Lynis in ihrem jeweiligen Einsatzzweck einordnen kannst.

Kapitel 8: Linux Privilege Escalation

Lernziele

- Du findest ausnutzbare SUID-Binaries und sudo-Fehlkonfigurationen auf server2.
- Du nutzt ein automatisiertes Enumeration-Skript und bewertest seine Funde.
- Du dokumentierst einen vollständigen Weg von eingeschränktem Zugang zu root.

1. Einführung

Band 1, Kapitel 3 hat SUID und sudo als Konzepte eingeführt. Dieses Kapitel setzt voraus, dass bereits ein eingeschränkter Zugang zu `server2` besteht (etwa über den in Kapitel 3/4 gefundenen FTP-Dienst), und zeigt den Weg zu vollständigen Rechten.

2. Technische Grundlagen

Rechteausweitung nutzt Fehlkonfigurationen, die ein eingeschränktes Konto mehr tun lassen, als vorgesehen: ein SUID-Programm mit unnötiger Mächtigkeit, eine zu großzügige sudo-Regel, ein beschreibbares, von root ausgeführtes Skript.

Beispiel 8.1: SUID-Bestand und sudo-Rechte auf server2 prüfen

```
$ find / -perm -4000 -type f 2>/dev/null
$ sudo -l
```

3. Fachbegriffe

GTFOBins ist eine öffentliche Sammlung dokumentierter Missbrauchsmöglichkeiten gewöhnlicher Unix-Programme mit SUID- oder sudo-Rechten. **LinPEAS** automatisiert die Suche nach genau solchen Fehlkonfigurationen.

4. Praxis und Konfiguration

Beispiel 8.2: LinPEAS auf server2 ausführen

```
$ wget https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh
$ chmod +x linpeas.sh
$ ./linpeas.sh | tee linpeas_ausgabe.txt
```

Beispiel 8.3: Einen gefundenen Weg nachvollziehen (Konzept, GTFOBins-Prinzip)

```
sudo -l zeigt: (root) NOPASSWD: /usr/bin/find
$ sudo find . -exec /bin/sh -p \; -quit
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Jede Technik dieses Kapitels gehört ausschließlich gegen `server2` in diesem Labor. Rechteausweitung setzt bereits einen bestehenden, unautorisierten Zugriff voraus, wenn sie gegen ein fremdes System eingesetzt wird – ein eigenständiger, schwerer Rechtsverstoß.

6. Praktische Laborübungen

Übung A: Enumeration mit LinPEAS

Aufgabenstellung: Führe LinPEAS auf `server2` aus und identifiziere den aussichtsreichsten Fund. **Ausgangssituation:** Eingeschränkter Zugang zu `server2` (z. B. über FTP, Kapitel 3/4). **Voraussetzungen:** Internetzugang auf `security-client` zum Herunterladen von LinPEAS. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 8.2. **Erwartetes Ergebnis:** Eine farblich markierte Ausgabe mit mindestens einem rot hervorgehobenen Fund. **Überprüfung:** Der Fund lässt sich einer der bekannten Kategorien (SUID, sudo, Cron) zuordnen. **Fehleranalyse:** Keine Funde? Prüfe, ob `server2` tatsächlich unverändert nach Vorwort-Installation ist. **Musterlösung:** Eine dokumentierte, priorisierte Liste der wichtigsten LinPEAS-Funde.

Übung B: Vollständige Rechteausweitung durchführen und dokumentieren

Aufgabenstellung: Nutze den Fund aus Übung A, um vollständige root-Rechte zu erlangen, und dokumentiere den Weg. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** Beispiel 8.3 entsprechend dem eigenen Fund anpassen und ausführen. **Erwartetes Ergebnis:** `id` zeigt `uid=0(root)`. **Überprüfung:** Der vollständige Weg lässt sich von einem Dritten anhand der Dokumentation nachvollziehen. **Fehleranalyse:** Kein Erfolg? GTFOBins-Eintrag für das konkrete, gefundene Programm gezielt nachschlagen. **Musterlösung:** Eine vollständige, nachvollziehbare Dokumentation von Ausgangszugang bis root-Shell.

7. Typische Fehler

Fehler	Ursache	Behebung
LinPEAS-Ausgabe nicht priorisiert	zu viele Funde gleichzeitig behandelt	zuerst rot markierte, dann gelb markierte Funde prüfen
GTFOBins-Weg falsch übertragen	Syntax nicht an eigenes System angepasst	Eintrag exakt gegen die eigene Umgebung prüfen
Weg nicht dokumentiert	Zwischenschritte als „selbstverständlich" übersprungen	jeden Schritt von Anfang bis Ende festhalten

8. Zusammenfassung

Zusammenfassung

- Rechteausweitung nutzt Fehlkonfigurationen bei SUID, sudo oder automatisierten Abläufen.
- LinPEAS automatisiert die Suche nach solchen Fehlkonfigurationen.
- GTFOBins dokumentiert, wie gefundene Fehlkonfigurationen konkret ausgenutzt werden können.
- Jeder gefundene Weg sollte vollständig und nachvollziehbar dokumentiert werden.

9. Kontrollfragen

1. Was ist GTFOBins? (leicht)
2. Welches Werkzeug automatisiert die Suche nach Rechteausweitungs-Fehlkonfigurationen? (leicht)
3. Warum ist eine zu großzügige sudo-Regel ein typisches Einfallstor? (mittel)
4. Was bedeutet NOPASSWD in einer sudo-Regel? (mittel)
5. Du findest auf `server2` sowohl ein ausnutzbares SUID-Programm als auch eine sudo-Fehlkonfiguration. Welchen Weg dokumentierst du zuerst, und warum könnte die Reihenfolge für einen Bericht relevant sein? (schwer)

Lösungshinweise: 1. Eine öffentliche Sammlung dokumentierter Missbrauchsmöglichkeiten gewöhnlicher Unix-Programme. 2. LinPEAS. 3. Weil ein mächtiges, über sudo erlaubtes Programm oft einen direkten Weg zu einer vollständigen Shell bietet. 4. Dass der Befehl ohne erneute Passworteingabe ausgeführt werden darf. 5. Beide Wege sollten dokumentiert werden; die Reihenfolge im Bericht richtet sich sinnvollerweise nach Zuverlässigkeit und Einfachheit des jeweiligen Wegs, nicht nach Fundreihenfolge, damit ein Auftraggeber den am leichtesten zu behebenden oder kritischsten Weg zuerst sieht.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] LinPEAS erfolgreich gegen `server2` ausgeführt hast,
- [ ] einen gefundenen Weg zur Rechteausweitung erfolgreich genutzt hast,
- [ ] den vollständigen Weg nachvollziehbar dokumentiert hast,
- [ ] mehrere gefundene Wege sinnvoll priorisieren kannst.

Kapitel 9: Linux Rootkits und Malware

Lernziele

- Du erklärst den Unterschied zwischen einem Rootkit und gewöhnlicher Schadsoftware.
- Du setzt rkhunter zur Rootkit-Erkennung ein.
- Du erkennst typische Persistenzmechanismen nach einer erfolgreichen Rechteausweitung.

1. Einführung

Nach einer erfolgreichen Rechteausweitung (Kapitel 8) sichert sich ein tatsächlicher Angreifer häufig dauerhaften Zugriff. Dieses Kapitel zeigt, woran sich das erkennen lässt.

2. Technische Grundlagen

Ein **Rootkit** verbirgt gezielt Prozesse, Dateien oder Netzwerkverbindungen vor normalen Systemwerkzeugen, oft durch Manipulation des Kernels oder zentraler Systembibliotheken. Gewöhnliche Schadsoftware (Kapitel 12 in Band 2 zu ClamAV) muss sich nicht zwingend verbergen; ein Rootkit tut genau das als Kernfunktion.

Beispiel 9.1: rkhunter installieren und einen Lauf starten

```
$ sudo apt install rkhunter
$ sudo rkhunter --update
$ sudo rkhunter --check
```

3. Fachbegriffe

Persistenz bezeichnet Mechanismen, mit denen sich ein Angreifer nach einem Neustart oder einer Sitzungsbeendigung erneut Zugriff verschafft, etwa über einen manipulierten Cron-Job oder einen zusätzlichen SSH-Schlüssel. Ein **Rootkit-Scanner** vergleicht Systemzustände gegen bekannte Verhaltensmuster und Signaturen bekannter Rootkits.

4. Praxis und Konfiguration

Beispiel 9.2: Typische Persistenzmechanismen manuell prüfen

```
$ cat ~/.ssh/authorized_keys     # unbekannte, zusätzliche Schlüssel?
$ crontab -l                     # unbekannte, zusätzliche Einträge?
$ ls -la /etc/cron.d/
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Installiere niemals testweise ein echtes, funktionsfähiges Rootkit, auch nicht im eigenen Labor. Die Analyse in diesem Kapitel beschränkt sich bewusst auf Erkennungswerkzeuge und das Prüfen typischer, bekannter Persistenzmuster.

6. Praktische Laborübungen

Übung A: rkhunter gegen server2 ausführen

Aufgabenstellung: Führe rkhunter gegen `server2` aus und werte den Bericht aus. **Ausgangssituation:** `server2` nach Kapitel 8. **Voraussetzungen:** sudo-Rechte, Internetzugang für `--update`. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 9.1. **Erwartetes Ergebnis:** Ein vollständiger Bericht ohne bestätigte Rootkit-Funde (da keine echte Schadsoftware installiert wurde). **Überprüfung:** Der Bericht listet geprüfte Kategorien vollständig auf. **Fehleranalyse:** Warnungen ohne Rootkit-Bezug? Prüfe, ob es sich um allgemeine Konfigurationshinweise statt tatsächlicher Rootkit-Indikatoren handelt. **Musterlösung:** Ein vollständiger, dokumentierter rkhunter-Bericht.

Übung B: Persistenzmechanismen nach Übung aus Kapitel 8 prüfen

Aufgabenstellung: Prüfe `server2` nach den Übungen aus Kapitel 8 manuell auf eigene, versehentlich hinterlassene Spuren. **Ausgangssituation:** `server2` nach Kapitel 8, möglicherweise mit eigenen Teständerungen. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 9.2. **Erwartetes Ergebnis:** Eine vollständige Liste aller während der Übungen vorgenommenen Änderungen. **Überprüfung:** Jede gefundene Änderung lässt sich einer konkreten, eigenen Übung zuordnen. **Fehleranalyse:** Unerklärliche Einträge gefunden? Snapshot-Historie der VM prüfen, ob eine frühere Übung nicht vollständig zurückgesetzt wurde. **Musterlösung:** Eine vollständige Übersicht eigener Testspuren, als Übung in genau der Systematik, die auch ein echter forensischer Blick (Kapitel 16) verlangt.

7. Typische Fehler

Fehler	Ursache	Behebung
rkhunter-Warnung unreflektiert als Rootkit-Fund gewertet	fehlende Unterscheidung zwischen Hinweis und Bestätigung	jede Warnung einzeln inhaltlich prüfen
Echtes Rootkit zu Testzwecken installiert	Neugier ohne Risikoabwägung	ausschließlich Erkennungswerkzeuge und dokumentierte Muster nutzen
Eigene Testspuren mit echtem Angriff verwechselt	fehlende Dokumentation eigener Übungen	jede eigene Änderung während der Übungen protokollieren

8. Zusammenfassung

Zusammenfassung

- Ein Rootkit verbirgt sich aktiv, anders als gewöhnliche Schadsoftware.
- rkhunter prüft gegen bekannte Rootkit-Muster und -Signaturen.
- Persistenzmechanismen wie zusätzliche SSH-Schlüssel oder Cron-Einträge sind klassische Indikatoren.
- Echte Rootkits gehören niemals, auch nicht testweise, in ein Übungslabor.

9. Kontrollfragen

1. Was unterscheidet ein Rootkit von gewöhnlicher Schadsoftware? (leicht)
2. Welches Werkzeug wurde in diesem Kapitel zur Rootkit-Erkennung genutzt? (leicht)

3. Was bedeutet Persistenz im Kontext eines Angriffs? (mittel)
4. Nenne zwei manuell prüfbare Persistenzindikatoren. (mittel)
5. Warum sollte selbst im eigenen, isolierten Labor kein echtes Rootkit installiert werden? (schwer)

Lösungshinweise: 1. Ein Rootkit verbirgt sich aktiv vor normalen Systemwerkzeugen, gewöhnliche Schadsoftware muss das nicht zwingend. 2. rkhunter. 3. Mechanismen, mit denen sich ein Angreifer nach einem Neustart oder einer Sitzungsbeendigung erneut Zugriff verschafft. 4. Zusätzliche SSH-Schlüssel in `authorized_keys`, unbekannte Cron-Einträge. 5. Weil ein Rootkit potenziell den Kernel manipuliert und damit auch in einem isolierten Labor unkontrollierbare, schwer rückgängig zu machende Nebenwirkungen auf das Wirtssystem oder die Virtualisierungsumgebung haben kann.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] rkhunter erfolgreich ausgeführt und den Bericht gelesen hast,
- [ ] typische Persistenzmechanismen manuell geprüft hast,
- [ ] den Unterschied zwischen Rootkit und gewöhnlicher Schadsoftware erklären kannst,
- [ ] erklären kannst, warum echte Rootkits im Labor tabu bleiben.

Kapitel 10: Passwortsicherheit und Hashverfahren

Lernziele

- Du erklärst, warum Hashing statt Verschlüsselung für Passwörter verwendet wird.
- Du misst den Geschwindigkeitsunterschied zwischen schwachen und starken Hash-Verfahren.
- Du extrahierst Passwort-Hashes aus einer Linux-Systemdatei autorisiert.

1. Einführung

Band 1, Kapitel 10 hat Hashing von Verschlüsselung abgegrenzt. Dieses Kapitel vertieft das praktisch, als Grundlage für die Werkzeuge in Kapitel 11 und 12.

2. Technische Grundlagen

Ein Passwort-Hash lässt sich nicht zurückrechnen; ein System prüft ein eingegebenes Passwort, indem es erneut hasht und vergleicht. Moderne Verfahren wie bcrypt oder yescrypt sind absichtlich rechenintensiv, um massenhaftes Ausprobieren zu verlangsamen.

Beispiel 10.1: Verwendetes Hash-Verfahren eines Kontos erkennen

```
$ sudo grep testuser /etc/shadow
```

Das Präfix `$y$` kennzeichnet yescrypt (Debian-Standard seit einigen Jahren), `$6$` SHA-512-Crypt, `$2b$` bcrypt.

3. Fachbegriffe

Salting fügt jedem Passwort vor dem Hashen eine individuelle, zufällige Zeichenfolge hinzu, damit identische Passwörter unterschiedliche Hashes ergeben und vorberechnete Tabellen (Rainbow Tables) wirkungslos werden. `unshadow` kombiniert `/etc/passwd` und `/etc/shadow` zu einem für Cracking-Werkzeuge lesbaren Format.

4. Praxis und Konfiguration

Beispiel 10.2: Hashes für die Werkzeuge in Kapitel 11/12 vorbereiten (nur eigene Testkonten!)

```
$ sudo unshadow /etc/passwd /etc/shadow > testhashes.txt
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Führe `unshadow` ausschließlich auf einem System aus, dessen `/etc/shadow` du mit eigener, autorisierter Administratorberechtigung einsehen darfst. `/etc/shadow` enthält die Hashes aller Systemkonten; unbefugter Zugriff darauf ist für sich genommen bereits strafbar.

6. Praktische Laborübungen

Übung A: Hash-Verfahren von server1 und server2 vergleichen

Aufgabenstellung: Ermittle das verwendete Hash-Verfahren eines Testkontos auf beiden Servern. **Ausgangssituation:** Testkonten auf `server1` und `server2` vorhanden. **Voraussetzungen:** sudo-Rechte auf beiden Systemen. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 10.1, auf beiden Servern. **Erwartetes Ergebnis:** Beide zeigen dasselbe, aktuelle Standardverfahren, da beide dieselbe Debian-13-Basis nutzen. **Überprüfung:** Das Präfix stimmt mit der Dokumentation des verwendeten Verfahrens überein. **Fehleranalyse:** Unterschiedliche Präfixe trotz gleicher Debian-Version? Prüfe, ob eines der Konten mit einem abweichenden PAM-Modul angelegt wurde. **Musterlösung:** Eine Tabelle mit Kontoname, System und erkanntem Hash-Präfix.

Übung B: Testhashes für die folgenden Kapitel vorbereiten

Aufgabenstellung: Erzeuge eine `testhashes.txt` nach Beispiel 10.2 ausschließlich mit eigenen Testkonten. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** sudo-Rechte. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 10.2. **Erwartetes Ergebnis:** Eine Datei im John-the-Ripper-kompatiblen Format, bereit für Kapitel 11. **Überprüfung:** Die Datei enthält ausschließlich die zuvor selbst angelegten Testkonten, keine realen Systemkonten. **Fehleranalyse:** Datei leer? Prüfe, ob tatsächlich sudo-Rechte für den Zugriff auf `/etc/shadow` vorlagen. **Musterlösung:** Eine vollständige, ausschließlich Testkonten enthaltende Hash-Datei.

7. Typische Fehler

Fehler	Ursache	Behebung
Hashing mit Verschlüsselung verwechselt	Begriffe klingen ähnlich	Hashing ist nicht umkehrbar, Verschlüsselung schon
Reale Systemkonten in Testhashes übernommen	fehlende Eingrenzung	ausschließlich eigens angelegte Testkonten verwenden
Salting-Zweck missverstanden	Begriff nur auswendig gelernt, nicht verstanden	Beispiel 10.1 mit zwei identischen Testpasswörtern auf verschiedenen Konten nachvollziehen

8. Zusammenfassung

Zusammenfassung

- Passwörter werden gehasht, nicht verschlüsselt, weil keine Rückrechnung nötig oder gewollt ist.
- Salting macht identische Passwörter zu unterschiedlichen Hashes.
- Moderne Verfahren wie yescrypt sind absichtlich rechenintensiv.
- `unshadow` bereitet Hashes für Cracking-Werkzeuge vor, ausschließlich mit eigener Berechtigung.

9. Kontrollfragen

1. Warum werden Passwörter gehasht statt verschlüsselt? (leicht)
2. Welches Hash-Präfix kennzeichnet yescrypt? (leicht)
3. Wozu dient Salting? (mittel)

4. Was macht `unshadow`? (mittel)
5. Zwei Testkonten mit identischem Passwort zeigen unterschiedliche Hash-Werte. Ist das ein Fehler? Begründe. (schwer)

Lösungshinweise: 1. Weil beim Login nur ein Vergleich nötig ist, keine Rückgewinnung des ursprünglichen Passworts. 2. `$y$`. 3. Es verhindert, dass identische Passwörter denselben Hash ergeben, und macht vorberechnete Tabellen wirkungslos. 4. Es kombiniert `/etc/passwd` und `/etc/shadow` zu einem für Cracking-Werkzeuge lesbaren Format. 5. Kein Fehler, sondern das erwartete, korrekte Verhalten von Salting: Jedes Konto erhält einen individuellen Salt, wodurch identische Passwörter unterschiedliche Hashes ergeben.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] das Hash-Verfahren eines Kontos anhand des Präfix erkennst,
- [ ] den Zweck von Salting an einem eigenen Beispiel nachvollzogen hast,
- [ ] mit `unshadow` eine Testhash-Datei autorisiert erzeugt hast,
- [ ] Hashing und Verschlüsselung klar unterscheiden kannst.

Kapitel 11: Sicherheitsanalyse mit John the Ripper

Lernziele

- Du prüfst Testhashes mit Wortlisten und Regeln.
- Du misst, wie stark Regeln die Trefferquote gegenüber einer reinen Wortliste erhöhen.
- Du bewertest die Passwortqualität von Testkonten anhand der Ergebnisse.

1. Einführung

Kapitel 10 hat Testhashes vorbereitet. **John the Ripper** prüft, wie widerstandsfähig die zugrunde liegenden Passwörter tatsächlich sind – ein praktischer Stresstest für die Passwort-Richtlinie aus Band 1, Kapitel 4.

2. Technische Grundlagen

John the Ripper probiert systematisch Kandidaten aus einer Wortliste (optional durch Regeln erweitert) gegen die vorbereiteten Hashes und vergleicht die Ergebnisse.

Beispiel 11.1: Grundlegender Wortlisten-Angriff

```
$ john --wordlist=/usr/share/wordlists/rockyou.txt testhashes.txt
```

3. Fachbegriffe

Regeln wandeln jeden Wortlisteneintrag automatisch in plausible Varianten um (Großschreibung, angehängte Ziffern). Die **Potfile** sammelt alle bereits gefundenen Passwörter.

4. Praxis und Konfiguration

Beispiel 11.2: Mit Regeln kombinieren und Ergebnisse ansehen

```
$ john --wordlist=/usr/share/wordlists/rockyou.txt --rules testhashes.txt
$ john --show testhashes.txt
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Nutze ausschließlich die in Kapitel 10 autorisiert erzeugten Testhashes. Das Knacken fremder, nicht autorisiert erlangter Passwort-Hashes ist unabhängig vom Ergebnis strafbar.

Die aus Band 1, Kapitel 4 bekannte Passwort-Richtlinie (`pam_pwquality`) lässt sich hier praktisch verifizieren: Testkonten, die nach dieser Richtlinie angelegt wurden, sollten deutlich widerstandsfähiger sein als absichtlich schwach gewählte Vergleichskonten.

6. Praktische Laborübungen

Übung A: Wortlisten-Angriff mit und ohne Regeln vergleichen

Aufgabenstellung: Teste dieselben Testhashes einmal ohne, einmal mit Regeln. **Ausgangssituation:** `testhashes.txt` aus Kapitel 10. **Voraussetzungen:** John the Ripper installiert, Wortliste verfügbar. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 11.1, dann 11.2. **Erwartetes Ergebnis:** Der Regel-gestützte Lauf findet mehr oder schneller Treffer als der reine Wortlisten-Lauf. **Überprüfung:** `john --show` listet die gefundenen Passwörter nach beiden Läufen. **Fehleranalyse:** Kein Unterschied? Prüfe, ob die Testpasswörter zu einfach (bereits in der reinen Wortliste enthalten) gewählt wurden. **Musterlösung:** Eine Gegenüberstellung der Trefferquote und -zeit beider Läufe.

Übung B: Passwort-Richtlinie aus Band 1 verifizieren

Aufgabenstellung: Lege ein Testkonto nach der Band-1-Richtlinie (`pam_pwquality`, mindestens zwölf Zeichen, gemischte Zeichenklassen) an und prüfe, ob John the Ripper es innerhalb einer angemessenen Zeit findet. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** Testkonto mit konformem Passwort anlegen, neue `testhashes.txt` erzeugen, Übung A wiederholen mit zeitlicher Begrenzung. **Erwartetes Ergebnis:** Das konforme Passwort wird innerhalb der Testzeit nicht gefunden. **Überprüfung:** `john --show` zeigt für dieses Konto keinen Treffer. **Fehleranalyse:** Passwort dennoch gefunden? Prüfen, ob es zufällig doch in einer Standardwortliste vorkommt. **Musterlösung:** Ein dokumentierter Beleg, dass die Band-1-Richtlinie gegen den Testlauf standhält.

7. Typische Fehler

Fehler	Ursache	Behebung
Testhashes mit realen Zugangsdaten vermischt	fehlende Trennung	ausschließlich eigens angelegte Testkonten verwenden
Kein Unterschied zwischen Regel-Läufen erkennbar	zu einfache Testpasswörter	Testpasswörter bewusst unterschiedlich stark wählen
Potfile zwischen Tests nicht beachtet	bereits gefundene Passwörter erscheinen sofort wieder	Potfile-Inhalt bei der Auswertung berücksichtigen

8. Zusammenfassung

Zusammenfassung

- John the Ripper prüft systematisch Wortlisten-Kandidaten, optional mit Regeln erweitert, gegen Passwort-Hashes.
- Regeln erhöhen die Trefferquote gegenüber einer reinen Wortliste deutlich.
- Die Passwort-Richtlinie aus Band 1 lässt sich praktisch gegen einen echten Cracking-Versuch verifizieren.
- Jeder Test beschränkt sich ausschließlich auf eigene, autorisiert erzeugte Testhashes.

9. Kontrollfragen

1. Was macht John the Ripper mit einer Wortliste? (leicht)
2. Wozu dienen Regeln? (leicht)

3. Warum erhöhen Regeln die Trefferquote gegenüber einer reinen Wortliste? (mittel)
4. Was zeigt `john --show`? (mittel)
5. Ein nach Band-1-Richtlinie angelegtes Testpasswort wird dennoch schnell gefunden. Was ist die wahrscheinlichste Erklärung, und wie prüfst du sie? (schwer)

Lösungshinweise: 1. Es probiert jeden Eintrag systematisch gegen die vorbereiteten Hashes. 2. Sie wandeln Wortlisteneinträge automatisch in plausible, menschentypische Varianten um. 3. Weil sie zusätzliche, realistische Varianten abdecken, die in der reinen Liste fehlen. 4. Bereits gefundene Passwörter aus der Potfile, ohne den Lauf erneut zu starten. 5. Wahrscheinlich enthält das gewählte, formal konforme Passwort trotzdem ein in Wortlisten verbreitetes Muster (z. B. ein Wort mit angehängter Jahreszahl); prüfen lässt sich das, indem man das konkrete Passwort gezielt in der verwendeten Wortliste sucht.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen Wortlisten-Angriff mit und ohne Regeln durchgeführt hast,
- [ ] den Unterschied in der Trefferquote erklären kannst,
- [ ] eine Band-1-konforme Passwort-Richtlinie praktisch verifiziert hast,
- [ ] ausschließlich mit eigenen Testhashes gearbeitet hast.

Kapitel 12: Sicherheitsanalyse mit Hashcat

Lernziele

- Du führst einen Benchmark durch und vergleichst die Geschwindigkeit verschiedener Hash-Verfahren.
- Du setzt einen maskenbasierten Angriff gegen ein bekanntes Passwortmuster ein.
- Du ordnest Hashcat gegenüber John the Ripper für dieses Labor ein.

1. Einführung

John the Ripper (Kapitel 11) ist CPU-zentriert. **Hashcat** verfolgt denselben Zweck GPU-beschleunigt und nutzt eine etwas andere, durchnummerierte Syntax.

2. Technische Grundlagen

Beispiel 12.1: Geschwindigkeit verschiedener Verfahren messen

```
$ hashcat -b -m 1000    # NTLM
$ hashcat -b -m 1800    # sha512crypt
```

Der Geschwindigkeitsunterschied zwischen beiden Benchmarks belegt praktisch, warum Band 1, Kapitel 10 moderne, absichtlich langsame Verfahren empfiehlt.

3. Fachbegriffe

Ein **maskenbasierter Angriff** (`-a 3`) probiert systematisch Zeichenkombinationen nach einem definierten Muster, effizienter als ein vollständiger Brute-Force über den gesamten Zeichenraum. Der **Hash-Modus** (`-m`) muss bei Hashcat, anders als oft bei John the Ripper, explizit angegeben werden.

4. Praxis und Konfiguration

Beispiel 12.2: Maskenbasierter Angriff gegen ein vermutetes Muster

```
$ hashcat -a 3 -m 1800 testhashes.txt 'Passwort?d?d?d?d'
```

Diese Maske testet „Passwort" gefolgt von vier Ziffern, etwa einer Jahreszahl.

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Wie in Kapitel 11 ausschließlich eigene, autorisiert erzeugte Testhashes verwenden. Die GPU-Beschleunigung macht Hashcat bei geeigneten Verfahren deutlich schneller, was die Versuchung erhöht, den Rahmen des eigenen Labors zu verlassen – die rechtliche Grenze bleibt dieselbe.

6. Praktische Laborübungen

Übung A: Benchmark durchführen und interpretieren

Aufgabenstellung: Führe Beispiel 12.1 aus und ordne die Ergebnisse praktisch ein. **Ausgangssituation:** Hashcat installiert, keine GPU zwingend erforderlich (CPU-Modus möglich). **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 12.1. **Erwartetes Ergebnis:** NTLM zeigt eine deutlich höhere Hashrate als sha512crypt. **Überprüfung:** Der Unterschied beträgt mindestens eine Größenordnung. **Fehleranalyse:** Kein GPU-Gerät erkannt? Mit `-D 1` gezielt den CPU-Modus erzwingen. **Musterlösung:** Eine dokumentierte Gegenüberstellung beider Benchmark-Werte mit Einordnung.

Übung B: Maskenbasierten Angriff gegen ein Testmuster einsetzen

Aufgabenstellung: Lege ein Testkonto mit einem Passwort nach dem Muster aus Beispiel 12.2 an und knacke es gezielt. **Ausgangssituation:** Ergebnis aus Übung A, neue Testhashes nach Kapitel 10 erzeugt. **Voraussetzungen:** wie Übung A. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 12.2. **Erwartetes Ergebnis:** Ein Treffer innerhalb kurzer Zeit, deutlich schneller als ein vollständiger Brute-Force über denselben Zeichenraum. **Überprüfung:** `hashcat -m 1800 testhashes.txt --show` zeigt das gefundene Passwort. **Fehleranalyse:** Kein Treffer? Prüfe, ob das Testpasswort tatsächlich exakt dem Maskenmuster entspricht. **Musterlösung:** Ein dokumentierter, erfolgreicher Maskenangriff mit Laufzeitangabe.

7. Typische Fehler

Fehler	Ursache	Behebung
Falscher Hash-Modus angegeben	Format nicht vorher geprüft	mit `--example-hashes -m <nummer>` gegenprüfen
Maske passt nicht exakt zum Testpasswort	Länge oder Zeichensatz falsch angenommen	Maske an die tatsächliche Struktur anpassen
Benchmark-Werte unreflektiert verglichen	Hardware-Unterschiede nicht beachtet	eigene Werte nur gegen eigene, konsistente Hardware vergleichen

8. Zusammenfassung

Zusammenfassung

- Hashcat misst GPU-beschleunigt dieselbe Grundaufgabe wie John the Ripper.
- Der Geschwindigkeitsunterschied zwischen Hash-Verfahren bestätigt praktisch die Empfehlungen aus Band 1.
- Maskenbasierte Angriffe sind bei bekanntem Muster deutlich effizienter als vollständiger Brute-Force.
- Dieselbe rechtliche Grenze wie bei John the Ripper gilt unverändert.

9. Kontrollfragen

1. Was misst `hashcat -b`? (leicht)
2. Welche Option wählt den maskenbasierten Angriffsmodus? (leicht)
3. Warum muss der Hash-Modus bei Hashcat explizit angegeben werden? (mittel)

4. Wie lässt sich der CPU-Modus bei Hashcat erzwingen? (mittel)
5. Für welches Szenario eignet sich ein maskenbasierter Angriff besser als ein Wortlisten-Angriff, und warum? (schwer)

Lösungshinweise: 1. Die Geschwindigkeit (Hashes pro Sekunde) für ein gewähltes Verfahren. 2. `-a 3`. 3. Weil Hashcat anders als John the Ripper das Format nicht zuverlässig automatisch erkennt. 4. Mit der Option `-D 1`. 5. Wenn ein konkretes, bekanntes oder vermutetes Muster vorliegt (z. B. ein Grundwort mit Jahreszahl), da die Maske gezielt nur diesen begrenzten Raum durchsucht statt eine vollständige Wortliste.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen Benchmark durchgeführt und interpretiert hast,
- [ ] einen maskenbasierten Angriff erfolgreich eingesetzt hast,
- [ ] Hashcat und John the Ripper in ihrem jeweiligen Einsatzzweck einordnest,
- [ ] ausschließlich mit eigenen Testhashes gearbeitet hast.

Kapitel 13: Grundlagen von Metasploit

Lernziele

- Du nutzt die Metasploit-Datenbank zur Verwaltung von Scan-Ergebnissen.
- Du führst ein Auxiliary-Modul gegen server2 aus.
- Du setzt einen passenden, dokumentierten Exploit testweise gegen server2 ein.

1. Einführung

Nmap (Kapitel 3) und Wireshark (Kapitel 4) liefern Informationen. **Metasploit** bündelt tausende Module, um aus diesen Informationen gezielt einen Zugang zu erlangen – ein vollständiges Exploitation-Framework.

2. Technische Grundlagen

Metasploit organisiert Scan-Ergebnisse und Funde in **Workspaces**, gestützt auf eine lokale Datenbank.

Beispiel 13.1: Workspace anlegen und Nmap-Scan einbinden

```
$ msfconsole -q
msf6 > workspace -a band3
msf6 > db_nmap -sV 192.168.56.30
```

3. Fachbegriffe

Ein **Auxiliary-Modul** führt Hilfsaufgaben wie Scanning oder Login-Versuche ohne eigenen Exploit aus. Ein **Exploit-Modul** nutzt eine konkrete Schwachstelle, ein **Payload** läuft danach auf dem Zielsystem.

4. Praxis und Konfiguration

Beispiel 13.2: Anonymen FTP-Zugang mit Auxiliary prüfen

```
msf6 > use auxiliary/scanner/ftp/anonymous
msf6 auxiliary(anonymous) > set RHOSTS 192.168.56.30
msf6 auxiliary(anonymous) > run
```

Beispiel 13.3: Einen passenden Exploit suchen und einsetzen (Konzept)

```
msf6 > search type:exploit <erkannter Dienst>
msf6 > use <gefundenes Modul>
msf6 exploit(...) > set RHOSTS 192.168.56.30
msf6 exploit(...) > check
msf6 exploit(...) > run
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Setze Exploit-Module ausschließlich gegen `server2` in diesem isolierten Labor ein. `check` vor `run` nutzen, wo verfügbar, um Fehlalarme zu vermeiden, bevor ein invasiverer Schritt folgt.

6. Praktische Laborübungen

Übung A: Workspace und Auxiliary-Scan einrichten

Aufgabenstellung: Richte einen Workspace ein und prüfe `server2` auf anonymen FTP-Zugang. **Ausgangssituation:** `server2` mit FTP-Dienst aus dem Vorwort. **Voraussetzungen:** Metasploit mit aktiver Datenbank (`db_status`). **Schritt-für-Schritt-Anleitung:** siehe Beispiel 13.1, 13.2. **Erwartetes Ergebnis:** Eine Bestätigung, ob anonymer Zugang möglich ist. **Überprüfung:** `hosts` und `services` zeigen `server2` mit dem geprüften Dienst. **Fehleranalyse:** Keine Datenbankverbindung? `msfdb init` erneut ausführen. **Musterlösung:** Ein dokumentiertes Ergebnis des Auxiliary-Laufs im Workspace.

Übung B: Einen passenden Exploit gegen server2 einsetzen

Aufgabenstellung: Finde anhand der bisherigen Scan-Ergebnisse (Kapitel 3) ein passendes Exploit-Modul und setze es gegen `server2` ein. **Ausgangssituation:** Ergebnis aus Übung A sowie Kapitel 3. **Voraussetzungen:** ein auf `server2` tatsächlich laufender, bekanntermaßen verwundbarer Dienst. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 13.3. **Erwartetes Ergebnis:** Eine Sitzung mit Zugriff auf `server2`. **Überprüfung:** `sessions -l` zeigt die aktive Sitzung. **Fehleranalyse:** `check` meldet „not vulnerable"? Versionsstand des Dienstes gegenprüfen. **Musterlösung:** Eine dokumentierte, erfolgreiche Sitzung mit Angabe von Modul, Zielsystem und erreichtem Rechteniveau.

7. Typische Fehler

Fehler	Ursache	Behebung
Kein Workspace angelegt, Ergebnisse verstreut	Datenbankfunktion nicht genutzt	immer zuerst Workspace anlegen
Exploit ohne vorheriges `check` direkt ausgeführt	Ungeduld	`check` nutzen, wo verfügbar
Modul gegen falsche IP-Adresse gerichtet	Verwechslung server1/server2	Zieladresse vor jedem Lauf prüfen

8. Zusammenfassung

Zusammenfassung

- Metasploit bündelt Scanning, Exploitation und Post-Exploitation in einem Framework mit Datenbank-Unterstützung.
- Auxiliary-Module prüfen ohne eigenen Exploit, Exploit-Module nutzen konkrete Schwachstellen aktiv aus.
- `check` reduziert das Risiko von Fehlalarmen vor einem invasiveren Schritt.
- Jeder Einsatz bleibt auf `server2` im eigenen Labor beschränkt.

9. Kontrollfragen

1. Wozu dient ein Workspace in Metasploit? (leicht)

2. Was ist der Unterschied zwischen Auxiliary- und Exploit-Modulen? (leicht)
3. Wozu dient der Befehl `check`? (mittel)
4. Welcher Befehl bindet einen Nmap-Scan direkt in die Datenbank ein? (mittel)
5. Ein Exploit-Modul meldet nach `check` „target is not exploitable". Welche zwei möglichen Ursachen prüfst du zuerst? (schwer)

Lösungshinweise: 1. Er trennt Hosts, Dienste und Funde verschiedener Projekte innerhalb derselben Installation. 2. Auxiliary führt Hilfsaufgaben ohne eigenen Exploit aus, Exploit nutzt eine konkrete Schwachstelle aktiv aus. 3. Er prüft Verwundbarkeit, ohne den eigentlichen Exploit auszulösen. 4. `db_nmap`. 5. Erstens, ob die tatsächlich installierte Version des Zieldienstes von der im Modul vorausgesetzten Version abweicht; zweitens, ob die Zieladresse (`RHOSTS`) tatsächlich korrekt gesetzt ist.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen Workspace angelegt und einen Nmap-Scan eingebunden hast,
- [ ] ein Auxiliary-Modul erfolgreich eingesetzt hast,
- [ ] einen passenden Exploit erfolgreich gegen `server2` eingesetzt hast,
- [ ] den Unterschied zwischen Auxiliary- und Exploit-Modulen erklären kannst.

Kapitel 14: Webserver- und Anwendungssicherheit

Lernziele

- Du findest eine SQL-Injection-Schwachstelle in einer Testanwendung.
- Du nutzt Burp Suite, um eine Anfrage systematisch zu verändern.
- Du ordnest einen gefundenen Fehler der OWASP-Systematik zu.

1. Einführung

Band 2, Kapitel 15 hat Webserver-Härtung behandelt. Dieses Kapitel prüft aus Angreiferperspektive, ob eine Webanwendung auf `server2` tatsächlich verwundbar ist.

2. Technische Grundlagen

Burp Suite schaltet sich als Proxy zwischen Browser und Webanwendung und macht jede HTTP-Anfrage sichtbar, veränderbar und wiederholbar. Eine **SQL-Injection** entsteht, wenn eine Anwendung Nutzereingaben ungeprüft in eine Datenbankabfrage einfügt.

Beispiel 14.1: Eine verdächtige Anfrage im Repeater testen

```
GET /suche?id=1' HTTP/1.1
Host: 192.168.56.30
```

3. Fachbegriffe

Der **Repeater** sendet eine einzelne Anfrage gezielt wiederholt, mit jeweils eigenen Änderungen. Die **OWASP Top 10** kategorisieren die kritischsten Web-Risiken; SQL-Injection fällt unter die Kategorie Injection.

4. Praxis und Konfiguration

Beispiel 14.2: Eine Union-basierte Injection testen

```
GET /suche?id=1' UNION SELECT benutzer, passwort FROM konten -- - HTTP/1.1
Host: 192.168.56.30
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Teste diese Techniken ausschließlich gegen eine auf `server2` eigens dafür installierte Testanwendung (z. B. DVWA), niemals gegen eine reale Anwendung ohne Auftrag.

6. Praktische Laborübungen

Übung A: Eine SQL-Injection im Repeater nachweisen

Aufgabenstellung: Teste die Anfrage aus Beispiel 14.1/14.2 gegen eine Testanwendung auf `server2`. **Ausgangssituation:** Eine verwundbare Testanwendung (z. B. DVWA) auf `server2` installiert. **Voraussetzungen:** Burp Suite auf `security-client` eingerichtet. **Schritt-für-Schritt-Anleitung:** Anfrage im Proxy abfangen, an Repeater senden, Parameter nach Beispiel 14.2 verändern. **Erwartetes Ergebnis:** Die Antwort enthält Daten, die über die ursprünglich vorgesehene Funktion hinausgehen. **Überprüfung:** Die Antwortlänge oder der Inhalt weicht sichtbar vom regulären Ergebnis ab. **Fehleranalyse:** Keine Veränderung sichtbar? Prüfe, ob der Parametername tatsächlich mit dem der Anwendung übereinstimmt. **Musterlösung:** Ein dokumentierter Fund mit Anfrage, Antwort und Erklärung der Schwachstelle.

Übung B: Fund nach OWASP einordnen

Aufgabenstellung: Ordne den Fund aus Übung A der passenden OWASP-Kategorie zu und formuliere eine Gegenmaßnahme. **Ausgangssituation:** Ergebnis aus Übung A. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** OWASP-Top-10-Liste konsultieren, passende Kategorie wählen, Gegenmaßnahme formulieren. **Erwartetes Ergebnis:** Eine kurze, vollständige Zuordnung mit Gegenmaßnahme. **Überprüfung:** Die genannte Gegenmaßnahme (z. B. parametrisierte Abfragen) behebt das Problem strukturell. **Fehleranalyse:** Falsche Kategorie gewählt? Fundbeschreibung noch einmal gegen die Kategorie-Definitionen prüfen. **Musterlösung:** „Injection" als Kategorie, parametrisierte Abfragen als strukturelle Gegenmaßnahme.

7. Typische Fehler

Fehler	Ursache	Behebung
Injection-Test gegen reale Anwendung	fehlende Eingrenzung auf Testsystem	ausschließlich gegen `server2`-Testanwendungen arbeiten
Fund ohne Kategorie-Zuordnung dokumentiert	fehlender Bezug zu anerkanntem Standard	jeden Fund mit OWASP-Kategorie versehen
Gegenmaßnahme nur oberflächlich benannt	„Eingaben besser prüfen" ohne Konkretisierung	konkrete, strukturelle Maßnahme wie parametrisierte Abfragen nennen

8. Zusammenfassung

Zusammenfassung

- Burp Suite macht HTTP-Anfragen sichtbar, veränderbar und wiederholbar.
- SQL-Injection entsteht durch ungeprüfte Einbindung von Nutzereingaben in Datenbankabfragen.
- Jeder Fund sollte einer anerkannten Kategorie wie den OWASP Top 10 zugeordnet werden.
- Parametrisierte Abfragen lösen das Problem strukturell.

9. Kontrollfragen

1. Wozu dient der Burp-Suite-Repeater? (leicht)
2. Unter welche OWASP-Kategorie fällt SQL-Injection? (leicht)

3. Wie entsteht eine SQL-Injection grundsätzlich? (mittel)
4. Welche strukturelle Gegenmaßnahme behebt SQL-Injection zuverlässig? (mittel)
5. Ein gefundener Fehler lässt sich sowohl als Injection als auch als Broken Access Control einordnen. Wie entscheidest du? (schwer)

Lösungshinweise: 1. Er sendet eine einzelne Anfrage gezielt wiederholt mit jeweils eigenen Änderungen. 2. Injection. 3. Durch ungeprüfte Einbindung von Nutzereingaben in eine Datenbankabfrage. 4. Parametrisierte Abfragen (Prepared Statements). 5. Nach dem technischen Kern des Fehlers entscheiden: Liegt die Ursache in der fehlenden Trennung von Daten und Befehl (Injection), oder in einer fehlenden Berechtigungsprüfung (Access Control)? Beide Kategorien können bei komplexeren Fehlern auch gemeinsam zutreffen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] eine SQL-Injection mit Burp Suite erfolgreich nachgewiesen hast,
- [ ] den Fund korrekt einer OWASP-Kategorie zugeordnet hast,
- [ ] eine strukturelle Gegenmaßnahme formulieren kannst,
- [ ] den Unterschied zwischen Injection und Access-Control-Fehlern erklären kannst.

Kapitel 15: Incident Response

Lernziele

- Du durchläufst die sechs klassischen Incident-Response-Phasen an einem simulierten Vorfall.
- Du erstellst ein kurzes, im Ernstfall nutzbares Playbook.
- Du dokumentierst einen Vorfall chronologisch während seines Verlaufs.

1. Einführung

Alle vorherigen Kapitel haben Schwachstellen gefunden und ausgenutzt. Dieses Kapitel simuliert die Gegenseite: Wie reagiert man, wenn ein solcher Vorfall tatsächlich (simuliert) eintritt?

2. Technische Grundlagen

Die sechs Phasen Preparation, Identification, Containment, Eradication, Recovery und Lessons Learned strukturieren jede Reaktion auf einen Sicherheitsvorfall, unabhängig von seiner Art.

Beispiel 15.1: Phasenablauf am simulierten Vorfall (Konzept)

```
Identification -> auditd-Alarm (Band 2, Kapitel 10) zeigt Änderung an /etc/sudoers auf server2
Containment    -> betroffenes Konto temporär sperren
Eradication    -> Ursache klären (z. B. Kapitel 8/13 dieses Bandes als Ursprung)
Recovery       -> Konfiguration zurücksetzen, Konto reaktivieren
Lessons Learned -> sudoers-Regel enger fassen (Band 2, Kapitel 2)
```

3. Fachbegriffe

Ein **Playbook** ist eine im Voraus festgelegte, kurze Anleitung für ein wiederkehrendes Szenario. **Containment** stoppt die weitere Ausbreitung, bevor die eigentliche Ursache behoben wird.

4. Praxis und Konfiguration

Beispiel 15.2: Ein einfaches Vorfall-Protokoll führen

```
14:02 - auditd-Alarm: Änderung an /etc/sudoers auf server2
14:04 - Bestätigt: Änderung durch eigene Übung aus Kapitel 13
14:05 - Konfiguration zurückgesetzt, keine weitere Maßnahme nötig
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Dokumentation während eines Vorfalls ist zuverlässiger als eine nachträgliche Rekonstruktion aus dem Gedächtnis. Beginne das Protokoll mit dem ersten Hinweis, nicht erst nach der Behebung.

6. Praktische Laborübungen

Übung A: Einen simulierten Vorfall vollständig durchlaufen

Aufgabenstellung: Nutze die aus Kapitel 13 erlangte Sitzung auf `server2` als simulierten Vorfall und durchlaufe alle sechs Phasen. **Ausgangssituation:** Ergebnis aus Kapitel 13, auditd-Regeln aus Band 2 aktiv. **Voraussetzungen:** Zugriff auf `server2` als Administrator zur Reaktion.
Schritt-für-Schritt-Anleitung: siehe Beispiel 15.1, jede Phase schriftlich dokumentieren. **Erwartetes Ergebnis:** Eine vollständige, chronologische Dokumentation aller sechs Phasen. **Überprüfung:** Jede Phase ist klar von der vorherigen abgegrenzt und begründet. **Fehleranalyse:** Phasen vermischt? Containment (Sofortmaßnahme) von Eradication (Ursachenbehebung) sauber trennen. **Musterlösung:** Eine vollständige, nachvollziehbare Dokumentation von der Erkennung bis zur Lehre für die Zukunft.

Übung B: Ein Playbook für Brute-Force-Alarme erstellen

Aufgabenstellung: Erstelle ein kurzes Playbook für den Fall eines Fail2ban-Alarms (Band 2, Kapitel 7) auf `server1`. **Ausgangssituation:** Fail2ban aus Band 2 aktiv. **Voraussetzungen:** keine.
Schritt-für-Schritt-Anleitung: Playbook mit fünf Schritten nach dem Muster aus Band 2, Kapitel 8 formulieren. **Erwartetes Ergebnis:** Ein kurzes, im Ernstfall tatsächlich nutzbares Dokument.
Überprüfung: Eine dritte Person könnte dem Playbook ohne Rückfragen folgen. **Fehleranalyse:** Zu lang oder zu vage? Auf die tatsächlich notwendigen Entscheidungsschritte kürzen. **Musterlösung:** Ein fünfstufiges, konkretes Playbook mit klaren Entscheidungskriterien.

7. Typische Fehler

Fehler	Ursache	Behebung
Containment und Eradication vermischt	fehlende Phasentrennung	Sofortmaßnahme von Ursachenbehebung sauber trennen
Dokumentation erst nach Abschluss erstellt	Zeitdruck während des Vorfalls	Protokoll ab dem ersten Hinweis parallel führen
Playbook zu allgemein formuliert	fehlende Konkretisierung	auf das konkrete Szenario und System zuschneiden

8. Zusammenfassung

Zusammenfassung

- Die sechs Incident-Response-Phasen strukturieren jede Reaktion auf einen Sicherheitsvorfall.
- Containment stoppt die Ausbreitung, Eradication behebt die Ursache.
- Ein gutes Playbook ist kurz, konkret und unter Zeitdruck nutzbar.
- Dokumentation beginnt mit dem ersten Hinweis, nicht erst nach der Behebung.

9. Kontrollfragen

1. Nenne die sechs Incident-Response-Phasen. (leicht)
2. Was unterscheidet Containment von Eradication? (leicht)
3. Warum sollte Dokumentation während, nicht nach einem Vorfall beginnen? (mittel)
4. Was macht ein Playbook im Ernstfall nutzbar? (mittel)

5. Ein Alarm stellt sich nach der Untersuchung als eigene, autorisierte Testaktivität heraus. Welche Phasen durchläufst du trotzdem noch, und warum? (schwer)

Lösungshinweise: 1. Preparation, Identification, Containment, Eradication, Recovery, Lessons Learned. 2. Containment stoppt die Ausbreitung sofort, Eradication beseitigt die zugrunde liegende Ursache. 3. Weil eine nachträgliche Rekonstruktion aus dem Gedächtnis Details wie genaue Zeitpunkte verliert. 4. Kürze, Konkretheit und klare, unter Zeitdruck befolgbare Entscheidungsschritte. 5. Lessons Learned durchläufst du trotzdem, da sich daraus etwa die Erkenntnis ergeben kann, Testaktivitäten künftig vorab anzukündigen, um unnötige Alarmierung zu vermeiden.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen simulierten Vorfall vollständig durch alle sechs Phasen begleitet hast,
- [ ] ein kurzes, nutzbares Playbook erstellt hast,
- [ ] einen Vorfall chronologisch während seines Verlaufs dokumentiert hast,
- [ ] erklären kannst, warum Lessons Learned auch bei einem Fehlalarm sinnvoll ist.

Kapitel 16: Forensische Grundlagen

Lernziele

- Du sicherst flüchtige Daten eines Systems in der richtigen Reihenfolge.
- Du erstellst ein forensisches Abbild eines Datenträgers mit Prüfsumme.
- Du dokumentierst eine lückenlose Chain of Custody für gesicherte Beweise.

1. Einführung

Nach Eradication und Recovery (Kapitel 15) bleibt oft eine letzte Frage: Was ist genau passiert, und lässt sich das gerichtsfest belegen? Dieses Kapitel vertieft die forensische Nachbereitung.

2. Technische Grundlagen

Forensische Sicherung folgt der **Order of Volatility**: flüchtige Daten (Arbeitsspeicher, aktive Netzwerkverbindungen) zuerst, dauerhaftere Daten (Festplatteninhalt) danach, da flüchtige Daten beim Abschalten des Systems unwiederbringlich verloren gehen.

Beispiel 16.1: Order of Volatility (von flüchtig zu dauerhaft)

```
1. Arbeitsspeicher und laufende Prozesse
2. Aktive Netzwerkverbindungen
3. Temporäre Dateisysteme
4. Festplatteninhalt
5. Archivierte Backups
```

3. Fachbegriffe

Ein **forensisches Abbild** (Image) ist eine vollständige, bitgenaue Kopie eines Datenträgers, erstellt mit einer zugehörigen Prüfsumme zum späteren Nachweis der Unverändertheit. Die **Chain of Custody** dokumentiert lückenlos, wer wann auf ein Beweismittel zugegriffen hat.

4. Praxis und Konfiguration

Beispiel 16.2: Flüchtigen Zustand sichern (Konzept, vor jeder Abschaltung)

```
$ ps auxf > prozesse.txt
$ ss -tulpen > verbindungen.txt
$ sudo cat /proc/meminfo > speicher.txt
```

Beispiel 16.3: Ein forensisches Abbild mit Prüfsumme erstellen

```
$ sudo dd if=/dev/sdb of=/forensik/server2_image.dd bs=4M status=progress
$ sha256sum /forensik/server2_image.dd > server2_image.sha256
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Arbeite forensisch ausschließlich mit Kopien, niemals direkt mit dem Originaldatenträger. Jede Veränderung am Original, selbst ein versehentliches Einbinden, kann spätere Beweiskraft zerstören.

6. Praktische Laborübungen

Übung A: Flüchtige Daten in korrekter Reihenfolge sichern

Aufgabenstellung: Sichere den flüchtigen Zustand von `server2` nach dem simulierten Vorfall aus Kapitel 15. **Ausgangssituation:** `server2` noch im Zustand nach Kapitel 13/15, nicht neu gestartet. **Voraussetzungen:** sudo-Rechte, bevor das System abgeschaltet wird. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 16.2, in der gezeigten Reihenfolge. **Erwartetes Ergebnis:** Drei Dateien mit Prozess-, Verbindungs- und Speicherinformationen zum Zeitpunkt der Sicherung. **Überprüfung:** Die gesicherten Prozesse stimmen mit der zuvor über `sessions -l` beobachteten Aktivität überein. **Fehleranalyse:** System bereits neu gestartet? Dieser Schritt dokumentiert dann exemplarisch, warum die Reihenfolge entscheidend ist. **Musterlösung:** Drei vollständige, mit Zeitstempel versehene Sicherungsdateien.

Übung B: Ein forensisches Abbild mit Prüfsumme erstellen

Aufgabenstellung: Erstelle ein Abbild einer Testpartition oder eines Datei-Images und bestätige seine Integrität. **Ausgangssituation:** Eine zusätzliche Testpartition oder ein Datei-Image, keine produktiv genutzten Daten. **Voraussetzungen:** ausreichend Speicherplatz für das Abbild.
Schritt-für-Schritt-Anleitung: siehe Beispiel 16.3. **Erwartetes Ergebnis:** Ein vollständiges Abbild mit zugehöriger Prüfsummendatei. **Überprüfung:** `sha256sum -c server2_image.sha256` bestätigt die Unverändertheit. **Fehleranalyse:** `dd` sehr langsam oder bricht ab? Blockgröße (`bs`) anpassen oder Testpartition verkleinern. **Musterlösung:** Ein verifiziertes, unverändertes forensisches Abbild samt Prüfsumme.

7. Typische Fehler

Fehler	Ursache	Behebung
Flüchtige Daten durch vorschnellen Neustart verloren	Order of Volatility nicht beachtet	immer flüchtige vor dauerhaften Daten sichern
Arbeit direkt am Originaldatenträger	fehlendes Bewusstsein für Beweiskraft	ausschließlich mit verifizierten Kopien arbeiten
Keine Prüfsumme erstellt	Schritt als „nicht notwendig" übersprungen	Prüfsumme immer unmittelbar nach dem Abbild erstellen

8. Zusammenfassung

Zusammenfassung

- Die Order of Volatility bestimmt, flüchtige vor dauerhaften Daten zu sichern.
- Ein forensisches Abbild ist eine bitgenaue Kopie mit zugehöriger Prüfsumme.
- Die Chain of Custody dokumentiert lückenlosen Zugriff auf Beweismittel.
- Forensische Arbeit erfolgt ausschließlich an Kopien, nie am Original.

9. Kontrollfragen

1. Was beschreibt die Order of Volatility? (leicht)
2. Wozu dient eine Prüfsumme bei einem forensischen Abbild? (leicht)
3. Warum sollte niemals direkt am Originaldatenträger gearbeitet werden? (mittel)
4. Was ist die Chain of Custody? (mittel)
5. Ein System muss aus betrieblichen Gründen sofort neu gestartet werden, bevor flüchtige Daten gesichert wurden. Welche Konsequenz hat das für eine spätere forensische Auswertung? (schwer)

Lösungshinweise: 1. Die Reihenfolge, in der Daten gesichert werden sollten, von flüchtig zu dauerhaft. 2. Sie belegt später, dass das Abbild seit der Erstellung unverändert geblieben ist. 3. Weil jede Veränderung am Original die spätere Beweiskraft zerstören kann. 4. Die lückenlose Dokumentation, wer wann auf ein Beweismittel zugegriffen hat. 5. Flüchtige Daten wie Arbeitsspeicherinhalt und aktive Verbindungen gehen unwiederbringlich verloren, was die spätere Rekonstruktion des genauen Tathergangs erheblich erschwert oder unmöglich macht.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] flüchtige Daten in korrekter Reihenfolge gesichert hast,
- [ ] ein forensisches Abbild mit Prüfsumme erstellt und verifiziert hast,
- [ ] die Chain of Custody erklären kannst,
- [ ] die Konsequenz eines vorschnellen Neustarts einschätzen kannst.

Kapitel 17: Security Auditing

Lernziele

- Du planst ein vollständiges Audit mit klar definiertem Scope und Methodik.
- Du führst ein strukturiertes Audit gegen server2 durch, das mehrere Kapitel dieses Bandes verbindet.
- Du bewertest Funde nach einem anerkannten Schweregrad-System.

1. Einführung

Die vorherigen Kapitel haben einzelne Werkzeuge behandelt. Ein **Security Audit** verbindet sie zu einem methodischen, vollständigen Prüfprozess, wie ihn Band 1, Kapitel 1 bereits als Konzept angedeutet hat.

2. Technische Grundlagen

Ein Audit durchläuft typischerweise Reconnaissance, Scanning, Schwachstellenanalyse, kontrollierte Ausnutzung und Bewertung, in dieser Reihenfolge, mit klar definiertem Scope (Kapitel 1).

Beispiel 17.1: Ablauf eines strukturierten Audits (Konzept)

```
1. Scope festlegen (Kapitel 1)
2. Reconnaissance und Scanning (Kapitel 3-5)
3. Automatisierte Schwachstellenanalyse (Kapitel 6-7)
4. Kontrollierte Ausnutzung bestätigter Funde (Kapitel 8, 13-14)
5. Schweregrad-Bewertung jedes Fundes
6. Bericht (Kapitel 18)
```

3. Fachbegriffe

CVSS (Common Vulnerability Scoring System) bewertet eine Schwachstelle anhand mehrerer Faktoren (Ausnutzbarkeit, Auswirkung) zu einem Wert zwischen 0 und 10. Ein **Finding** ist ein einzelner, dokumentierter Befund mit Beschreibung, Nachweis und Bewertung.

4. Praxis und Konfiguration

Beispiel 17.2: Ein Finding strukturiert dokumentieren

```
Titel:        Anonymer FTP-Zugang auf server2
Schweregrad:  Mittel (CVSS ca. 5.3)
Nachweis:     auxiliary/scanner/ftp/anonymous, erfolgreich (Kapitel 13)
Empfehlung:   Anonymen Zugang deaktivieren, Authentifizierung erzwingen
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Ein Audit ohne vorab definierten Scope wiederholt den in Kapitel 1 beschriebenen Fehler. Jedes Audit in diesem Buch bleibt auf `server1`, `server2` und `client1` im eigenen Labor beschränkt.

6. Praktische Laborübungen

Übung A: Ein vollständiges Audit gegen server2 planen

Aufgabenstellung: Erstelle einen schriftlichen Audit-Plan nach Beispiel 17.1 für `server2`. **Ausgangssituation:** Alle vorherigen Kapitel dieses Bandes abgeschlossen. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** Für jeden Schritt aus Beispiel 17.1 die konkret einzusetzenden Werkzeuge aus diesem Buch benennen. **Erwartetes Ergebnis:** Ein vollständiger, kapitelübergreifender Plan. **Überprüfung:** Jeder Schritt verweist auf ein konkretes, bereits in diesem Buch behandeltes Kapitel. **Fehleranalyse:** Lücke im Plan? Inhaltsverzeichnis dieses Bandes erneut konsultieren. **Musterlösung:** Ein vollständiger, sechsstufiger Plan mit konkreter Werkzeugzuordnung.

Übung B: Drei Findings mit CVSS-Einschätzung dokumentieren

Aufgabenstellung: Dokumentiere drei tatsächliche Funde aus den vorherigen Kapiteln nach dem Schema aus Beispiel 17.2. **Ausgangssituation:** Funde aus Kapitel 3-14 liegen vor. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** Drei unterschiedliche Funde auswählen, jeweils grob nach CVSS einschätzen (eine exakte Berechnung ist für diese Übung nicht erforderlich). **Erwartetes Ergebnis:** Drei vollständige, strukturierte Findings. **Überprüfung:** Jeder Schweregrad ist nachvollziehbar begründet. **Fehleranalyse:** Einschätzung wirkt willkürlich? Mit einem CVSS-Rechner im Internet gegenprüfen. **Musterlösung:** Drei vollständige Findings mit Titel, Schweregrad, Nachweis und Empfehlung.

7. Typische Fehler

Fehler	Ursache	Behebung
Audit ohne Scope begonnen	Kapitel 1 nicht konsequent angewendet	Scope immer vor Beginn schriftlich festlegen
Findings ohne konkreten Nachweis	Behauptung statt Beleg	jedes Finding mit konkretem, nachvollziehbarem Beleg versehen
Schweregrad willkürlich vergeben	kein anerkanntes System genutzt	CVSS oder vergleichbares System konsequent anwenden

8. Zusammenfassung

Zusammenfassung

- Ein Security Audit verbindet die Einzelwerkzeuge dieses Bandes zu einem methodischen Prozess.
- CVSS bewertet Schwachstellen nach einem anerkannten, vergleichbaren System.
- Jedes Finding braucht Titel, Schweregrad, Nachweis und Empfehlung.
- Der Scope aus Kapitel 1 gilt für jedes Audit unverändert.

9. Kontrollfragen

1. Wofür steht CVSS? (leicht)
2. Was gehört in ein vollständiges Finding? (leicht)
3. Warum sollte jedes Finding einen konkreten Nachweis enthalten? (mittel)
4. In welcher Reihenfolge läuft ein strukturiertes Audit typischerweise ab? (mittel)

5. Zwei Findings haben denselben CVSS-Wert, aber unterschiedliche Auswirkung auf den konkreten Geschäftsbetrieb. Sollten sie gleich priorisiert werden? (schwer)

Lösungshinweise: 1. Common Vulnerability Scoring System. 2. Titel, Schweregrad, Nachweis, Empfehlung. 3. Damit der Fund für Dritte nachvollziehbar und überprüfbar bleibt. 4. Scope, Reconnaissance/Scanning, Schwachstellenanalyse, kontrollierte Ausnutzung, Bewertung, Bericht. 5. Nicht zwingend: CVSS bewertet die technische Schwere, der tatsächliche Geschäftskontext (Schutzbedarf, Band 2, Kapitel 16) kann eine abweichende Priorisierung rechtfertigen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] einen vollständigen Audit-Plan erstellt hast,
- [ ] drei Findings mit CVSS-Einschätzung dokumentiert hast,
- [ ] den Unterschied zwischen technischer Schwere und Geschäftsauswirkung erklären kannst,
- [ ] ein Audit konsequent innerhalb eines definierten Scopes durchführst.

Kapitel 18: Professionelle Sicherheitsberichte

Lernziele

- Du gliederst einen Sicherheitsbericht nach anerkanntem Aufbau.
- Du schreibst eine Management Summary für ein nicht-technisches Publikum.
- Du erstellst einen vollständigen Abschlussbericht für das Audit aus Kapitel 17.

1. Einführung

Das beste Audit (Kapitel 17) bleibt wirkungslos, wenn sein Ergebnis nicht verständlich und handlungsleitend kommuniziert wird. Dieses letzte Kapitel schließt den Bogen der gesamten Reihe.

2. Technische Grundlagen

Ein professioneller Sicherheitsbericht richtet sich an zwei unterschiedliche Zielgruppen gleichzeitig: technisches Personal, das Findings im Detail nachvollziehen muss, und Entscheidungsträger, die eine knappe, verständliche Gesamteinschätzung brauchen.

Beispiel 18.1: Grundgliederung eines Sicherheitsberichts

```
1. Management Summary (ein bis zwei Seiten, nicht-technisch)
2. Scope und Methodik
3. Zusammenfassung der Findings nach Schweregrad
4. Detaillierte Findings (Kapitel 17, Beispiel 17.2)
5. Empfehlungen, priorisiert
6. Anhänge (Rohdaten, Scan-Ausgaben)
```

3. Fachbegriffe

Eine **Management Summary** fasst die wichtigsten Ergebnisse und Risiken ohne technischen Fachjargon zusammen, meist mit einer groben Risikoeinschätzung (hoch/mittel/niedrig) statt technischer Details. Eine **Executive Recommendation** benennt konkrete, priorisierte nächste Schritte für Entscheidungsträger.

4. Praxis und Konfiguration

Beispiel 18.2: Eine Management Summary (Auszug, Konzept)

```
Im Rahmen des vereinbarten Audits wurden drei Schwachstellen mit mittlerem
bis hohem Risiko auf server2 festgestellt, vor allem im Bereich unsicherer
Standardkonfiguration. server1 zeigte durch die vorab umgesetzte Härtung
deutlich weniger und weniger schwerwiegende Befunde. Eine priorisierte
Liste konkreter Maßnahmen findet sich in Abschnitt 5.
```

5. Sicherheitsbetrachtung

WARNUNG – Sicherheitsrisiko

Ein Sicherheitsbericht ist selbst ein sensibles Dokument: Er beschreibt im Detail, wie ein System verwundbar ist. Er gehört verschlüsselt übertragen, Zugriff darauf streng begrenzt, und nach Abschluss des Projekts gemäß vereinbarter Aufbewahrungsfrist gelöscht.

6. Praktische Laborübungen

Übung A: Management Summary schreiben

Aufgabenstellung: Schreibe eine Management Summary für das Audit aus Kapitel 17. **Ausgangssituation:** Drei Findings aus Kapitel 17, Übung B. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** siehe Beispiel 18.2 als Vorlage, mit den tatsächlichen eigenen Findings füllen. **Erwartetes Ergebnis:** Ein bis zwei Absätze, vollständig frei von technischem Fachjargon. **Überprüfung:** Eine fachfremde Person könnte die Kernaussage verstehen, ohne Rückfragen zu stellen. **Fehleranalyse:** Zu technisch geraten? Jeden Fachbegriff durch eine allgemeinverständliche Umschreibung ersetzen. **Musterlösung:** Eine kurze, verständliche Zusammenfassung mit grober Risikoeinschätzung.

Übung B: Vollständigen Abschlussbericht erstellen

Aufgabenstellung: Erstelle einen vollständigen Bericht nach der Gliederung aus Beispiel 18.1, der das gesamte Audit aus Kapitel 17 zusammenfasst. **Ausgangssituation:** Ergebnis aus Übung A sowie alle Funde dieses Bandes. **Voraussetzungen:** keine. **Schritt-für-Schritt-Anleitung:** Jeden Abschnitt aus Beispiel 18.1 mit den tatsächlichen Inhalten aus Kapitel 1-17 füllen. **Erwartetes Ergebnis:** Ein vollständiges, in sich stimmiges Berichtsdokument. **Überprüfung:** Der Bericht lässt sich ohne Rückgriff auf dieses Buch selbst eigenständig lesen und verstehen. **Fehleranalyse:** Abschnitt unvollständig? Entsprechendes Kapitel dieses Bandes erneut konsultieren. **Musterlösung:** Ein vollständiger, professioneller Abschlussbericht als Abschluss der gesamten Reihe.

7. Typische Fehler

Fehler	Ursache	Behebung
Management Summary zu technisch	Fachjargon unreflektiert übernommen	konsequent für fachfremdes Publikum schreiben
Bericht ohne Empfehlungen	nur Befunde, keine nächsten Schritte	jedem Finding eine konkrete Empfehlung zuordnen
Bericht ungeschützt versendet	Vertraulichkeit des Dokuments unterschätzt	verschlüsselt übertragen, Zugriff begrenzen

8. Zusammenfassung

Zusammenfassung

- Ein Sicherheitsbericht bedient technisches und nicht-technisches Publikum gleichzeitig.
- Die Management Summary fasst Ergebnisse verständlich und ohne Fachjargon zusammen.
- Jedes Finding braucht eine konkrete, priorisierte Empfehlung.
- Der Bericht selbst ist ein sensibles Dokument und verdient entsprechenden Schutz.

9. Kontrollfragen

1. An welche zwei Zielgruppen richtet sich ein Sicherheitsbericht typischerweise? (leicht)
2. Was gehört in eine Management Summary? (leicht)
3. Warum sollte eine Management Summary frei von Fachjargon sein? (mittel)
4. Warum ist ein Sicherheitsbericht selbst ein schützenswertes Dokument? (mittel)
5. Ein Auftraggeber möchte den Bericht nur als kurze E-Mail statt als vollständiges Dokument. Wie reagierst du professionell? (schwer)

Lösungshinweise: 1. Technisches Personal und nicht-technische Entscheidungsträger. 2. Eine kurze, verständliche Zusammenfassung der wichtigsten Ergebnisse und eine grobe Risikoeinschätzung. 3. Damit auch Entscheidungsträger ohne technischen Hintergrund die Kernaussage verstehen. 4. Weil er im Detail beschreibt, wie ein System verwundbar ist, und damit selbst ein attraktives Ziel darstellt. 5. Eine kurze Management Summary als E-Mail anbieten, aber auf den vollständigen, detaillierten Bericht als notwendige Grundlage für die technische Umsetzung der Empfehlungen bestehen.

10. Lernzielkontrolle

Du hast dieses Kapitel verstanden, wenn du:

- [ ] eine verständliche Management Summary geschrieben hast,
- [ ] einen vollständigen, eigenständig lesbaren Abschlussbericht erstellt hast,
- [ ] erklären kannst, warum ein Bericht selbst schützenswert ist,
- [ ] auf eine unangemessene Verkürzungsanfrage professionell reagieren kannst.

Abschlussprojekt Band 3: Linux Security Audit

Führe ein vollständiges, eigenständiges Sicherheitsaudit über das gesamte Vier-Maschinen-Labor durch:

1. **Informationssammlung:** Bedrohungsmodell (Kapitel 2), Netzwerkerkundung mit Nmap (Kapitel 3).
2. **Netzwerkanalyse:** Wireshark- und tcpdump-Mitschnitte relevanter Dienste (Kapitel 4-5).
3. **Schwachstellenbewertung:** Lynis und OpenSCAP gegen beide Server (Kapitel 6-7).
4. **Sicherheitsüberprüfung:** mindestens ein erfolgreicher Weg zur Rechteausweitung (Kapitel 8) und ein erfolgreicher Metasploit-Einsatz (Kapitel 13) gegen `server2`.
5. **Risikobewertung:** CVSS-eingeschätzte Findings für alle gefundenen Schwachstellen (Kapitel 17).
6. **Handlungsempfehlungen und Abschlussbericht:** ein vollständiger, zweiteiliger Bericht mit Management Summary und technischem Detailteil (Kapitel 18), der auf die in Band 2 bereits umgesetzte Härtung von `server1` ausdrücklich Bezug nimmt.

Dieses Projekt schließt die dreibändige Reihe: vom ersten, ungesicherten Server in Band 1 über seine professionelle Härtung in Band 2 bis zur eigenständigen, methodischen Überprüfung in Band 3.

Verzeichnis der Befehlsbeispiele

Beispiel V.1: Erweiterte Netzwerkarchitektur … Seite 8
Beispiel V.2: server2 einrichten (Kurzübersicht) … Seite 9
Beispiel 1.1: Derselbe Scan, drei rechtliche Einordnungen (Konzept) … Seite 10
Beispiel 1.2: Mindestinhalt eines Scope-Dokuments … Seite 10
Beispiel 2.1: Eine Vertrauensgrenze (Konzept) … Seite 13
Beispiel 2.2: STRIDE auf server2 angewendet (Auszug) … Seite 13
Beispiel 3.1: SYN-Scan mit Versionserkennung … Seite 16
Beispiel 3.2: Gezieltes NSE-Skript gegen einen erkannten Dienst … Seite 16
Beispiel 3.3: Ergebnis dokumentieren … Seite 16
Beispiel 4.1: Nur FTP-Verkehr zu server2 mitschneiden … Seite 19
Beispiel 4.2: Mitschnitt in der GUI analysieren … Seite 19
Beispiel 5.1: Verkehr direkt auf server2 mitschneiden … Seite 22
Beispiel 5.2: Verkehr direkt im Terminal lesbar anzeigen … Seite 22
Beispiel 5.3: Mitschnitt für spätere Analyse übertragen … Seite 22
Beispiel 6.1: Lynis installieren und ausführen … Seite 25
Beispiel 6.2: Ergebnisbericht einsehen … Seite 25
Beispiel 7.1: OpenSCAP installieren und verfügbare Profile anzeigen … Seite 28
Beispiel 7.2: Eine Prüfung gegen ein Standardprofil durchführen … Seite 28
Beispiel 8.1: SUID-Bestand und sudo-Rechte auf server2 prüfen … Seite 31
Beispiel 8.2: LinPEAS auf server2 ausführen … Seite 31
Beispiel 8.3: Einen gefundenen Weg nachvollziehen (Konzept, GTFOBins-Prinzip) … Seite 31
Beispiel 9.1: rkhunter installieren und einen Lauf starten … Seite 34
Beispiel 9.2: Typische Persistenzmechanismen manuell prüfen … Seite 34
Beispiel 10.1: Verwendetes Hash-Verfahren eines Kontos erkennen … Seite 37
Beispiel 10.2: Hashes für die Werkzeuge in Kapitel 11/12 vorbereiten (nur eigene Testkonten!) … Seite 37
Beispiel 11.1: Grundlegender Wortlisten-Angriff … Seite 40
Beispiel 11.2: Mit Regeln kombinieren und Ergebnisse ansehen … Seite 40
Beispiel 12.1: Geschwindigkeit verschiedener Verfahren messen … Seite 43
Beispiel 12.2: Maskenbasierter Angriff gegen ein vermutetes Muster … Seite 43
Beispiel 13.1: Workspace anlegen und Nmap-Scan einbinden … Seite 46
Beispiel 13.2: Anonymen FTP-Zugang mit Auxiliary prüfen … Seite 46
Beispiel 13.3: Einen passenden Exploit suchen und einsetzen (Konzept) … Seite 46
Beispiel 14.1: Eine verdächtige Anfrage im Repeater testen … Seite 49
Beispiel 14.2: Eine Union-basierte Injection testen … Seite 49
Beispiel 15.1: Phasenablauf am simulierten Vorfall (Konzept) … Seite 52
Beispiel 15.2: Ein einfaches Vorfall-Protokoll führen … Seite 52
Beispiel 16.1: Order of Volatility (von flüchtig zu dauerhaft) … Seite 55
Beispiel 16.2: Flüchtigen Zustand sichern (Konzept, vor jeder Abschaltung) … Seite 55
Beispiel 16.3: Ein forensisches Abbild mit Prüfsumme erstellen … Seite 55
Beispiel 17.1: Ablauf eines strukturierten Audits (Konzept) … Seite 58
Beispiel 17.2: Ein Finding strukturiert dokumentieren … Seite 58
Beispiel 18.1: Grundgliederung eines Sicherheitsberichts … Seite 61
Beispiel 18.2: Eine Management Summary (Auszug, Konzept) … Seite 61

Index

Burp Suite – 49, 50, 51
Chain of Custody – 55, 57
CVSS – 58, 59, 60, 63
Finding – 58, 59, 60, 61, 62, 63
GTFOBins – 31, 32, 33
Hashcat – 8, 43, 44, 45
John the Ripper – 8, 40, 41, 43, 45
LinPEAS – 31, 32, 33
Lynis – 25, 26, 27, 28, 30, 63
Management Summary – 61, 62, 63
Metasploit – 8, 46, 47, 63
Nmap – 8, 16, 17, 18, 19, 46, 48, 63
OpenSCAP – 28, 29, 30, 63
Order of Volatility – 55, 57
OWASP – 49, 50, 51
rkhunter – 34, 35, 36
Scope – 10, 11, 12, 58, 60
STRIDE – 13, 14, 15
tcpdump – 22, 23, 24, 63
Wireshark – 8, 19, 22, 23, 24, 46, 63